FIRST ASSIGNMENT REPORT

Hodgkin-Huxley Neuron Model

Name	: Jeremia Christ Immanuel Manalu
NRP	: 5023231017
Course	: Biocybernetics
Class	: Q
Lecturer	: Dr. Achmad Arifin, S.T., M.Eng.
Department	: Biomedical Engineering

FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS TECHNOLOGY

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2026

CHAPTER I. FUNDAMENTAL THEORY

1.1 The Hodgkin-Huxley Membrane and Its Equivalent Circuit

Hodgkin and Huxley described the excitable membrane of the squid giant axon as an electrical circuit in which the lipid bilayer acts as a capacitor and every ionic pathway acts as a conductance in series with a battery that represents the Nernst potential of that ion [1]. The circuit used in this assignment follows the week-9 lecture material [2] and is shown in Figure 1.1. The capacitive current J_C charges the membrane capacitance C_m, while the sodium, potassium and leakage currents J_{Na}, J_K and J_L flow through the conductances g_{Na}, g_K and g_L. A stimulus current density J_{in} is injected across the membrane. Because the patch is space clamped, the voltage is uniform over the membrane and Kirchhoff's current law applied to the single node of the circuit gives the current balance of Equation (1.1), from which the voltage equation (1.2) follows directly.

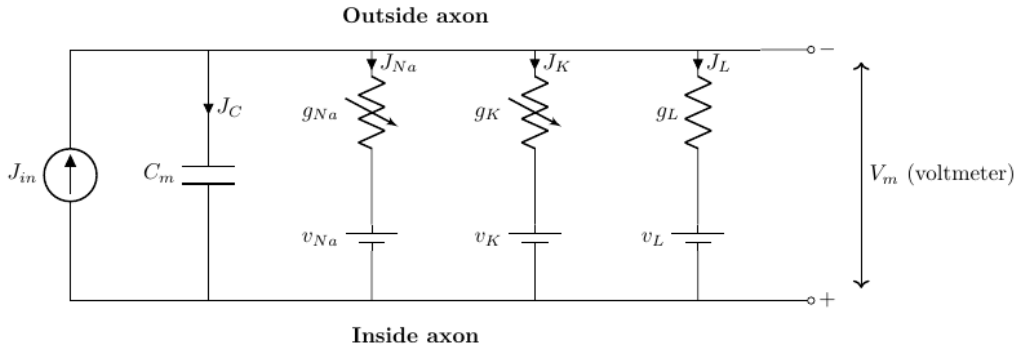


Figure 1.1. *Equivalent circuit of a space-clamped membrane patch in the lecturer's notation (rendered by the author after [1], [2])*

$$J_{in} = C_m \frac{dv}{dt} + J_{Na} + J_K + J_L \qquad (1.1)$$

$$\frac{dv}{dt} = \frac{1}{C_m}[J_{in} - (J_{Na} + J_K + J_L)] \qquad (1.2)$$

The ionic currents obey Ohm's law with a driving force equal to the difference between the membrane voltage and the reversal potential of each ion. Sodium and potassium conductances are not constant, Hodgkin and Huxley expressed them as a maximal conductance multiplied by powers of dimensionless gating variables m, h and n, whereas the leakage conductance is constant [1]:

$$J_{Na} = \bar{g}_{Na}\, m^3 h\, (v - v_{Na}), \qquad J_K = \bar{g}_K\, n^4\, (v - v_K), \qquad J_L = \bar{g}_L\, (v - v_L) \qquad (1.3)$$

A point that must be handled carefully is the sign convention. The original paper and the lecture material [1], [2] define v as the displacement of the membrane potential from rest with depolarization taken as negative, so that $v_{Na} = -115$ mV, $v_K = +12$ mV and $v_L = -10.613$ mV, and an inward (cathodal) stimulus is a negative J_{in}. Modern textbooks use the opposite

sign and absolute potentials. The two descriptions are related by a reflection and a shift, and the program reports the physiological membrane voltage through the alignment of Equation (1.4) with a resting potential of -70 mV. With this alignment the resting state $v = 0$ appears as $V_m = -70$ mV and an action potential appears as a positive excursion that peaks near $+35$ mV. The constants used throughout the work are those of the 1952 paper: $\bar{g}_{Na} = 120$, $\bar{g}_K = 36$ and $\bar{g}_L = 0.3$ mS/cm², $C_m = 1$ µF/cm², at the reference temperature of 6.3 °C.

$$V_m = -(v + 70) \text{ mV} \qquad (1.4)$$

1.2 Gating Kinetics and Rate Constants

Each gating variable represents the fraction of independent molecular gates that are in the permissive state and obeys first-order kinetics with voltage-dependent opening and closing rates $\alpha_x(v)$ and $\beta_x(v)$ [1]. Equation (1.5) can be written equivalently as a relaxation toward the steady state x_∞ with the time constant τ_x of Equation (1.6), which is the form used to interpret the phase plots in Chapter II.

$$\frac{dx}{dt} = \alpha_x(1 - x) - \beta_x x = -x(\alpha_x + \beta_x) + \alpha_x, \qquad x \in \{m, h, n\} \qquad (1.5)$$

$$x_\infty = \frac{\alpha_x}{\alpha_x + \beta_x}, \qquad \tau_x = \frac{1}{\alpha_x + \beta_x} \qquad (1.6)$$

The empirical rate functions fitted by Hodgkin and Huxley to their voltage-clamp data are listed in Equation (1.7) to (1.9) in the lecturer's sign convention, with v in mV and the rates in ms⁻¹ [1], [2]. Two of them, α_m and α_n, are of the form $a(v + b)/\left(e^{c(v+b)} - 1\right)$ and become an indeterminate $0/0$ at $v = -25$ mV and $v = -10$ mV respectively. These are removable singularities whose limits follow from L'Hôpital's rule, $\alpha_m(-25) = 1$ and $\alpha_n(-10) = 0.1$ ms⁻¹. The program evaluates both through the function $x/(e^x - 1)$ of Equation (1.10), which is replaced by its second-order Taylor series when $|x| < 10^{-4}$, instead of adding an arbitrary small number to the denominator.

$$\alpha_m = \frac{0.1\,(v + 25)}{e^{0.1v+2.5} - 1}, \qquad \beta_m = 4\,e^{v/18} \qquad (1.7)$$

$$\alpha_h = 0.07\,e^{v/20}, \qquad \beta_h = \frac{1}{e^{0.1v+3} + 1} \qquad (1.8)$$

$$\alpha_n = \frac{0.01\,(v + 10)}{e^{0.1v+1} - 1}, \qquad \beta_n = 0.125\,e^{v/80} \qquad (1.9)$$

$$\frac{x}{e^x - 1} \approx 1 - \frac{x}{2} + \frac{x^2}{12} \quad \text{for } |x| < 10^{-4} \qquad (1.10)$$

Temperature changes all rate constants by the same factor, written as $\phi = 3^{(T-6.3)/10}$ with T in °C, which corresponds to a Q_{10} of 3 [1]. Figure 1.2 shows the resulting steady states and time constants against the aligned membrane voltage. The sodium activation m is fast (its time constant is below 0.5 ms everywhere), whereas the sodium inactivation h and the

potassium activation n are slower by roughly an order of magnitude. This separation of time scales is the mechanism of the action potential: on depolarization m follows the voltage almost instantly and opens the sodium conductance, which depolarizes the membrane further, while the delayed closure of h and opening of n terminate the spike and produce the after-hyperpolarization and the refractory period.

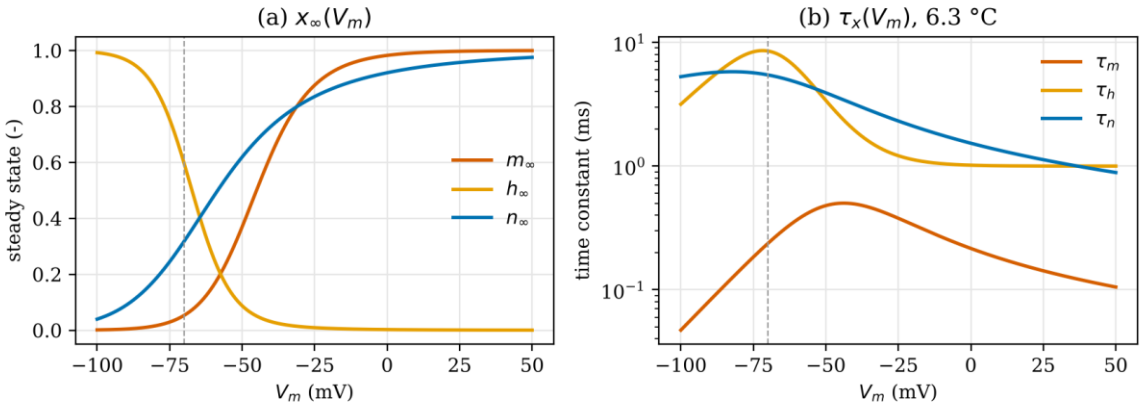


Figure 1.2. *(a) Steady-state values and (b) time constants of the gating variables computed from Equation (1.6) to (1.9) at 6.3 °C, the dashed line marks the resting potential (plotted by the author from the rate functions of [1])*

1.3 Fourth-Order Runge-Kutta Integration

The space-clamped model is a system of four coupled nonlinear ordinary differential equations for the state vector $\mathbf{Y} = [v, m, h, n]^T$, whose right-hand side $\mathbf{F}(t, \mathbf{Y})$ is formed by Equation (1.2) and three copies of Equation (1.5). The lecture material [3] integrates it with the classical fourth-order Runge-Kutta (RK4) method [4], which evaluates four slope estimates per step and combines them with Simpson-like weights:

$$\mathbf{K}_1 = \mathbf{F}(t_k, \mathbf{Y}_k), \qquad \mathbf{K}_2 = \mathbf{F}(t_k + \Delta t/2\,,\ \mathbf{Y}_k + \Delta t/2\,\mathbf{K}_1) \tag{1.11}$$

$$\mathbf{K}_3 = \mathbf{F}(t_k + \Delta t/2\,,\ \mathbf{Y}_k + \Delta t/2\,\mathbf{K}_2), \qquad \mathbf{K}_4 = \mathbf{F}(t_k + \Delta t,\ \mathbf{Y}_k + \Delta t\,\mathbf{K}_3) \tag{1.12}$$

$$\mathbf{Y}_{k+1} = \mathbf{Y}_k + \frac{\Delta t}{6}(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4) \tag{1.13}$$

The local truncation error of RK4 is of order Δt^5 and the global error of order Δt^4 [4]. With the step of 0.01 ms used here (the slide prints “10 sec”, which can only be 10 µs because the lecturer’s code works in milliseconds), the scheme resolves the sodium upstroke, whose time scale is about 0.1 ms, with ample margin. The lecturer’s source code [3] applies RK4 in a slightly different way: the rate constants are frozen at v_k, each gate is advanced by its own scalar RK4 step, and the voltage is then advanced with the new gate values held constant. This is an operator-splitting scheme that is first order in the coupling between voltage and gates. Both variants are implemented in the program so that the results can be compared directly with the lecturer’s program.

1.4 The Cable Equation of Action Potential Propagation

To describe propagation, the axon is divided into segments of length Δx that are connected by the internal (axoplasmic) and external resistances per unit length r_i and r_o, as in the transmission-line model of the lecture [2] shown in Figure 1.3 [5]. Every membrane element is either a passive RC element (c_m, g_m) or, in the active cable, the complete Hodgkin-Huxley circuit of Figure 1.1.

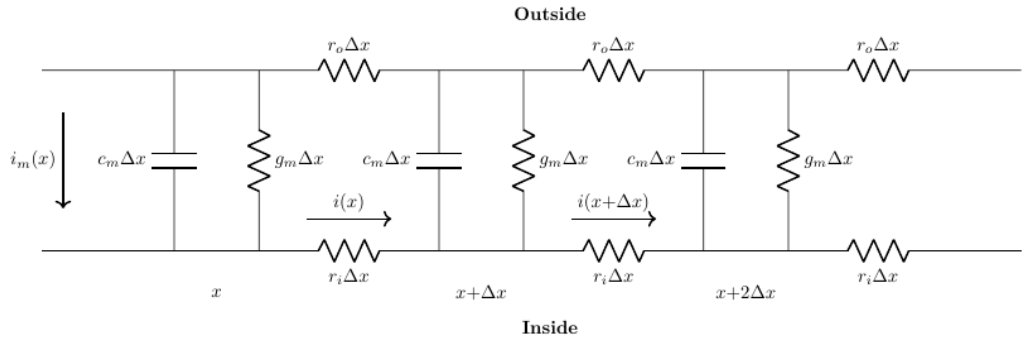


Figure 1.3. *Discretized axon as an RC transmission line (rendered by the author after [2], [5])*

Ohm's law along the axis and Kirchhoff's current law at a node give, in the limit $\Delta x \to 0$, the pair of equations (1.14) and (1.15), where i is the axial current and i_m the membrane current per unit length. Differentiating (1.14) with respect to x and substituting (1.15) eliminates the axial current and yields the cable equation (1.16). Its derivation is repeated step by step as part of Problem 2 in Chapter II.

$$\frac{\partial v}{\partial x} = -(r_o + r_i)i \qquad (1.14)$$

$$\frac{\partial i}{\partial x} = -i_m = -\left(c_m \frac{\partial v}{\partial t} + i_{ion}\right) \qquad (1.15)$$

$$\frac{\partial v}{\partial t} = D\,\frac{\partial^2 v}{\partial x^2} + \frac{J_{in} - J_{ion}}{C_m}, \qquad D = \frac{1}{(r_o + r_i)c_m} \qquad (1.16)$$

For a cylindrical axon of radius a and axoplasm resistivity R_i, the per-length quantities are $r_i = R_i/(\pi a^2)$, $c_m = 2\pi a\, C_m$ and $i_{ion} = 2\pi a\, J_{ion}$, so that $i_{ion}/c_m = J_{ion}/C_m$ [5]. In the passive cable $J_{ion} = g_m v$ and a steady input decays along the axon with the length constant $\lambda = [2\pi a\, g_m(r_o + r_i)]^{-1/2}$, such a cable cannot regenerate a signal. In the active cable $J_{ion} = J_{Na} + J_K + J_L$, so the depolarization that diffuses from an excited region into its neighbour opens the sodium conductance there and regenerates the full action potential. This is why an action potential travels at constant amplitude and constant velocity, and Hodgkin and Huxley showed that the velocity scales with $\sqrt{a}$ and computed 18.8 m/s for their squid axon at 18.5 °C, against 21.2 m/s measured [1]. Note that the slide prints the coefficient of the second derivative as $(r_o + r_i)C_m^{-1}$, the derivation gives $[(r_o + r_i)c_m]^{-1}$, which is also what the lecturer's source code computes.

1.5 Finite Difference Approximation and Stability

The cable equation is a parabolic partial differential equation of the diffusion type, the same type as the heat equation $u_t = \alpha u_{xx}$ used in the lecture to introduce its numerical solution [3]. On a grid $x_j = jh$, $t^n = nk$ with $v_j^n \approx v(x_j, t^n)$, the second spatial derivative is replaced by a central difference and the time derivative by a forward difference [4], [6]. The resulting explicit scheme, known as forward-time central-space (FTCS), computes every new value v_j^{n+1} from three known values of the previous time level, as illustrated by the stencil in Figure 1.4.

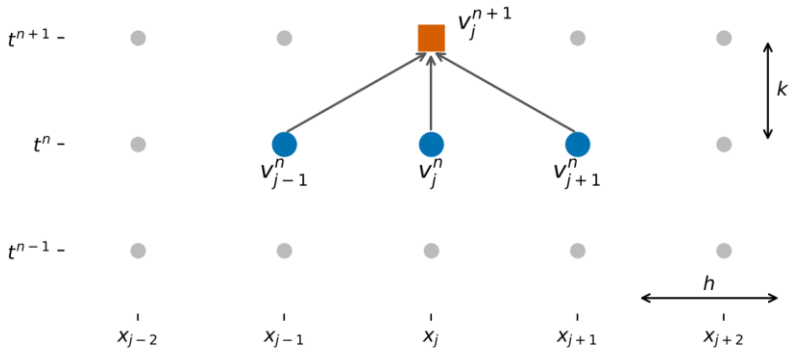


Figure 1.4. *Stencil of the explicit forward-time central-space scheme: the new value (square) is computed from three values of the previous time level (circles)*

The central difference has a truncation error of order h^2 and the forward difference of order k, so the scheme is consistent with an error of $O(k + h^2)$. Because it is explicit, it is only conditionally stable. A von Neumann analysis of the diffusion part shows that the amplification factor of a Fourier mode is $\xi = 1 - 4r\sin^2(\beta h/2)$ with the diffusion number $r = Dk/h^2$, and $|\xi| \leq 1$ for every wave number only if $r \leq 1/2$ [6]. The full derivation of the scheme and of this condition is given as the answer of Problem 2. The program checks the condition before every run and refuses to integrate when it is violated, reporting the largest admissible time step $k_{max} = h^2/(2D)$.

1.6 Refractory Period and Repetitive Firing

After an action potential the membrane cannot respond to a second stimulus for a short time and then responds only to a stronger one. During the absolute refractory period the sodium inactivation h is still small and the potassium activation n is still large, so no stimulus, however strong, can make the inward sodium current exceed the outward currents. During the relative refractory period h recovers and n decays with the time constants of Figure 1.2(b), and the threshold falls back toward its resting value [1]. Operationally, the refractory period is measured with a paired-pulse protocol: a conditioning pulse elicits an action potential and the smallest amplitude of a test pulse that still elicits a second action potential is determined as a function of the interval between the two pulses. A sustained depolarizing current, on the other hand, makes the membrane fire repetitively. After each spike the current recharges the

membrane toward threshold, and the larger the current, the faster the threshold is reached once the refractoriness has decayed. The input level therefore programs the inter-pulse interval and hence the firing frequency, up to the point where the membrane remains depolarized, sodium inactivation stays high and firing stops (depolarization block). These two behaviours are the subject of Problem 1 and are quantified in Section 2.3.

CHAPTER II. RESULT AND ANALYSIS

2.1 Problems Statement

1. Based on lecture material in week#9, develop a computer program to simulate Action Potential Generation (APG) using RK4 numerical integration. Describe how to align output from your program to fit the membrane voltage (V_m), show the refractory period, end how the Jin level can program inter-pulse interval.

2. Derive expression of integration of differential partial equation of Action Potential Propagation (APP) based on central difference, and forward difference method.

3. Develop a computer program to simulate the numerical integration method to show the propagation of AP by referring to materials of week#9. Describe how the model can exhibit the propagation.

4. From two scientific papers, write important aspects of HH model in these research works, and the impacts of HH works for advances technology for medical and biology.

A_model_of_the_pacemaking_neuron_of_the_respiratory_central_pattern_generator-1.pdf (28 October 2026, 12:38 PM)

High Speed and Low Digital Resources Im..-1.pdf (28 October 2026, 12:38 PM)

2.2 Answer and Code Explanation

2.2.1 Program Architecture

The program was developed in RAD Studio Delphi 13 Florence (VCL, Object Pascal) and is organized in three units. `HHModel.pas` is the numerical core and contains no user-interface code: it defines the model parameters and state records, the rate constants, both RK4 integrators, the stimulus protocols, the spike detector, the refractory and inter-pulse analyses and the cable solver. `Unit1.pas` with its form file `Unit1.dfm` is the main window, it reads the parameters from editable tables, calls the routines of `HHModel.pas` and draws the results with TeeChart. `EquationViewer.pas` opens every mathematical sheet (equivalent circuit, gating kinetics, RK4, cable equation, finite difference derivation and analysis definitions) in its own window with the caption “Jeremia Christ Immanuel Manalu - 5023231017 - ”, on these sheets, as in this report, the equations that are implemented in the code are highlighted in yellow. Keeping the numerics separate from the form made it possible to compile and regression-test `HHModel.pas` with Free Pascal against an independent Python implementation before it was used in the GUI.

The main window has six tabs: (1) APG with RK4, (2) gating phase plots, (3) refractory period, (4) Jin versus inter-pulse interval, (5) APP with the cable equation and (6) model parameters and equations. Figure 2.1 shows the first tab and Figure 2.2 one of the derivation windows that are opened from the main form. All routines listed below are complete and are

presented in the order of the problems in Section 2.1 and, within each problem, in the order in which they appear in the source files.

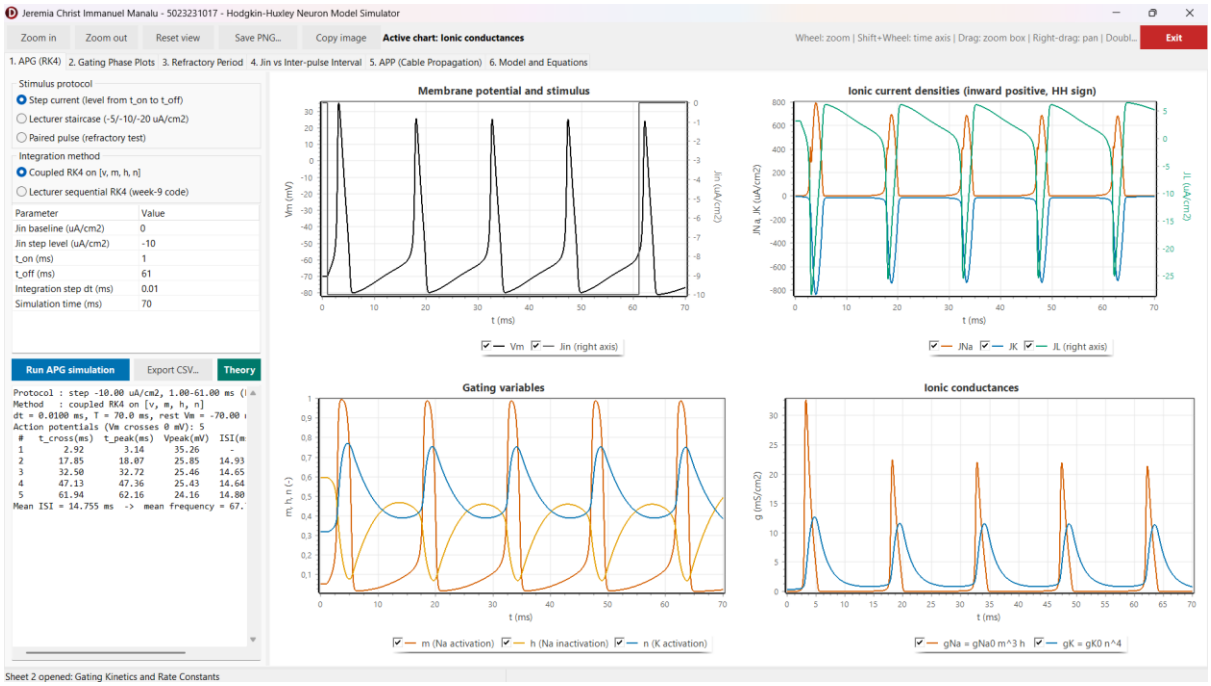


Figure 2.1. *Main window of the simulator, tab "1. APG (RK4)", after a step-current run*

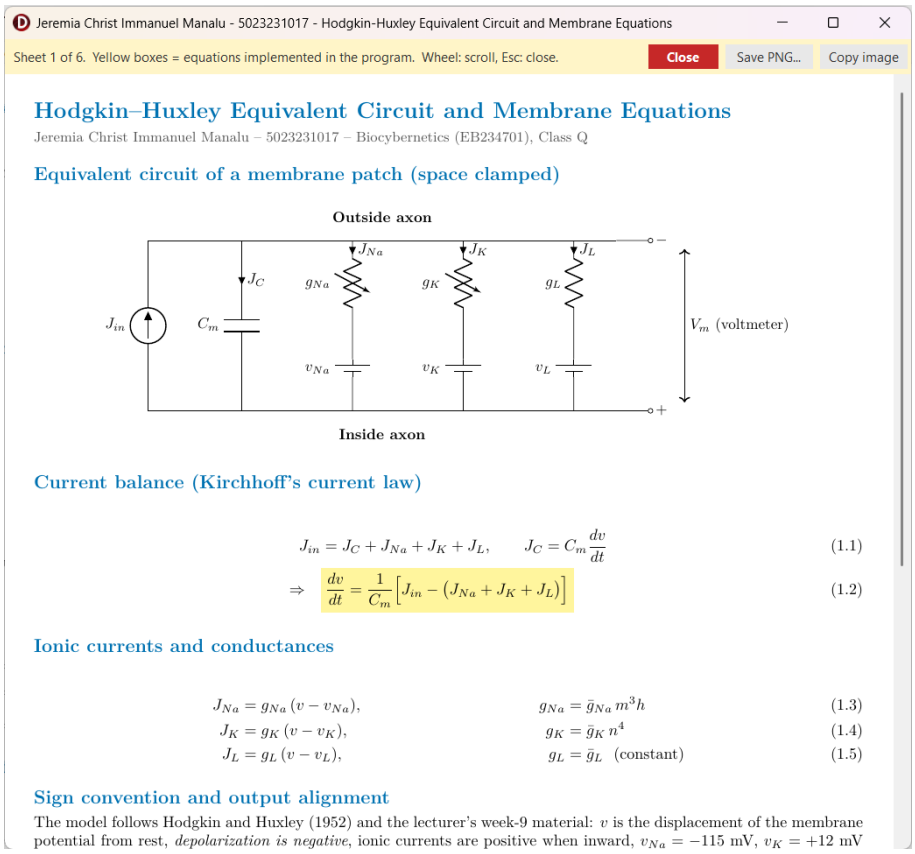


Figure 2.2. *Sheet window opened from the main form (sheet 1, equivalent circuit and membrane equations), yellow boxes mark equations implemented in the program*

2.2.2 Problem 1. Action Potential Generation with RK4

a. Model constants, rate constants and right-hand side

The routines below implement Equation (1.2) to (1.10). `DefaultHHParams` returns the constants of the 1952 paper in the lecturer's sign convention. `TemperatureFactor` computes $\phi = 3^{(T-6.3)/10}$, which multiplies every rate constant. `ExpRatio` evaluates $x/(e^x - 1)$ with the Taylor series near the removable singularity, and `ComputeRates` uses it to write α_m and α_n exactly: since $0.1(v + 25) = 0.1v + 2.5$, the numerator of α_m equals the exponent of its denominator, so $\alpha_m = \text{ExpRatio}(0.1v + 2.5)$, likewise $0.01(v + 10) = 0.1\,(0.1v + 1)$ gives $\alpha_n = 0.1\,\text{ExpRatio}(0.1v + 1)$. `SteadyState` returns x_∞ from Equation (1.6) and is used as the initial condition, so every run starts from the true resting state. `IonicCurrents` implements Equation (1.3), and `RestingConductance` returns the chord conductance $\bar{g}_{Na} m_\infty^3 h_\infty + \bar{g}_K n_\infty^4 + \bar{g}_L$ of the resting membrane, 0.6773 mS/cm², which is used as g_m of the passive cable in Problem 3 so that the passive and active cables have the same resting properties. `Derivatives` assembles $\mathbf{F}(t, \mathbf{Y})$ from Equation (1.2) and (1.5).

```
function DefaultHHParams: THHParams;
begin
  Result.GNa0 := 120.0;
  Result.GK0 := 36.0;
  Result.GL0 := 0.3;
  Result.VNa := -115.0;
  Result.VK := 12.0;
  Result.VL := -10.613;
  Result.Cm := 1.0;          // 0.001 mF/cm2 = 1 uF/cm2 (lecturer slide)
  Result.TempC := 6.3;       // phi = 1, rate equations as published
end;

function TemperatureFactor(const P: THHParams): Double;
begin
  // Q10 = 3 relative to the 6.3 deg C reference of Hodgkin & Huxley (1952
)
  Result := Power(3.0, (P.TempC - 6.3) / 10.0);
end;

function ExpRatio(const X: Double): Double;
begin
  // X / (exp(X) - 1) with its removable singularity at X = 0 (limit = 1).
  // A second-order Taylor expansion is used close to the singularity.
  if Abs(X) < 1.0E-4 then
    Result := 1.0 - 0.5 * X + X * X / 12.0
  else
    Result := X / (Exp(X) - 1.0);
end;

function ComputeRates(const V, Phi: Double): THHRates;
begin
  // alpha_m = 0.1(v+25)/(exp(0.1v+2.5)-1) = ExpRatio(0.1v+2.5)
  Result.Am := Phi * ExpRatio(0.1 * V + 2.5);
  Result.Bm := Phi * 4.0 * Exp(V / 18.0);
  Result.Ah := Phi * 0.07 * Exp(V / 20.0);
  Result.Bh := Phi * 1.0 / (Exp(0.1 * V + 3.0) + 1.0);
```

```
  // alpha_n = 0.01(v+10)/(exp(0.1v+1)-1) = 0.1 * ExpRatio(0.1v+1)
  Result.An := Phi * 0.1 * ExpRatio(0.1 * V + 1.0);
  Result.Bn := Phi * 0.125 * Exp(V / 80.0);
end;

function SteadyState(const V, Phi: Double): THHState;
var
  Rt: THHRates;
begin
  Rt := ComputeRates(V, Phi);
  Result.V := V;
  Result.M := Rt.Am / (Rt.Am + Rt.Bm);
  Result.H := Rt.Ah / (Rt.Ah + Rt.Bh);
  Result.N := Rt.An / (Rt.An + Rt.Bn);
end;

procedure IonicCurrents(const P: THHParams; const S: THHState;
  out JNa, JK, JL: Double);
begin
  JNa := P.GNa0 * S.M * S.M * S.M * S.H * (S.V - P.VNa);
  JK := P.GK0 * S.N * S.N * S.N * S.N * (S.V - P.VK);
  JL := P.GL0 * (S.V - P.VL);
end;

function RestingConductance(const P: THHParams): Double;
var
  S: THHState;
begin
  // Total (chord) membrane conductance of the resting membrane (mS/cm2);
  // used as gm of the equivalent passive cable.
  S := SteadyState(0.0, TemperatureFactor(P));
  Result := P.GNa0 * S.M * S.M * S.M * S.H + P.GK0 * Sqr(Sqr(S.N)) + P.GL0
;
end;

function Derivatives(const P: THHParams; const Phi: Double;
  const S: THHState; const Jin: Double): THHState;
var
  Rt: THHRates;
  JNa, JK, JL: Double;
begin
  Rt := ComputeRates(S.V, Phi);
  IonicCurrents(P, S, JNa, JK, JL);
  // Cm dv/dt = Jin - (JNa + JK + JL)
  Result.V := (Jin - (JNa + JK + JL)) / P.Cm;
  // dx/dt = alpha_x (1 - x) - beta_x x = -x(alpha_x + beta_x) + alpha_x
  Result.M := Rt.Am * (1.0 - S.M) - Rt.Bm * S.M;
  Result.H := Rt.Ah * (1.0 - S.H) - Rt.Bh * S.H;
  Result.N := Rt.An * (1.0 - S.N) - Rt.Bn * S.N;
end;
```

b. Fourth-order Runge-Kutta integration

`StepCoupledRK4` is a direct transcription of Equation (1.11) to (1.13) for the vector $\mathbf{Y} = [v, m, h, n]^T$, `AddScaled` forms the intermediate states $\mathbf{Y}_k + c\,\mathbf{K}$. The stimulus is evaluated at t_k, $t_k + \Delta t/2$ and $t_k + \Delta t$, which is required for fourth-order accuracy when J_{in} changes in time. `StepLecturerRK4` reproduces the week-9 code [3]: the rates are computed once at v_k,

`GateRK4` advances each gate with a scalar RK4 step of $dx/dt = \alpha(1-x) - \beta x$, and the voltage is advanced by a scalar RK4 step in which the new gate values are held constant. Writing the gate step for frozen α and β makes clear why this is a splitting scheme: the exact solution of the frozen gate equation is $x_{k+1} = x_\infty + (x_k - x_\infty)e^{-\Delta t/\tau_x}$, which RK4 reproduces to $O(\Delta t^5)$, but the voltage dependence of α and β within the step is neglected, which introduces a first-order coupling error. With $\Delta t = 0.01$ ms the two methods differ by at most 0.01 ms in spike times in all tested cases.

```
function AddScaled(const A, B: THHState; const C: Double): THHState; inlin
e;
begin
  // A + C * B
  Result.V := A.V + C * B.V;
  Result.M := A.M + C * B.M;
  Result.H := A.H + C * B.H;
  Result.N := A.N + C * B.N;
end;

function StepCoupledRK4(const P: THHParams; const Phi: Double;
  const S: THHState; const J0, JHalf, J1, Dt: Double): THHState;
var
  K1, K2, K3, K4: THHState;
begin
  // Classical 4th-order Runge-Kutta on Y = [v, m, h, n]:
  //   K1 = F(t, Y),            K2 = F(t + dt/2, Y + dt/2 K1)
  //   K3 = F(t + dt/2, Y + dt/2 K2), K4 = F(t + dt, Y + dt K3)
  //   Y(t+dt) = Y + dt/6 (K1 + 2K2 + 2K3 + K4)
  K1 := Derivatives(P, Phi, S, J0);
  K2 := Derivatives(P, Phi, AddScaled(S, K1, 0.5 * Dt), JHalf);
  K3 := Derivatives(P, Phi, AddScaled(S, K2, 0.5 * Dt), JHalf);
  K4 := Derivatives(P, Phi, AddScaled(S, K3, Dt), J1);
  Result.V := S.V + Dt / 6.0 * (K1.V + 2.0 * K2.V + 2.0 * K3.V + K4.V);
  Result.M := S.M + Dt / 6.0 * (K1.M + 2.0 * K2.M + 2.0 * K3.M + K4.M);
  Result.H := S.H + Dt / 6.0 * (K1.H + 2.0 * K2.H + 2.0 * K3.H + K4.H);
  Result.N := S.N + Dt / 6.0 * (K1.N + 2.0 * K2.N + 2.0 * K3.N + K4.N);
end;

function GateRK4(const X, A, B, Dt: Double): Double;
var
  K1, K2, K3, K4: Double;
begin
  // Scalar RK4 of dx/dt = A(1-x) - Bx with A, B frozen during the step
  K1 := Dt * (A * (1.0 - X) - B * X);
  K2 := Dt * (A * (1.0 - (X + 0.5 * K1)) - B * (X + 0.5 * K1));
  K3 := Dt * (A * (1.0 - (X + 0.5 * K2)) - B * (X + 0.5 * K2));
  K4 := Dt * (A * (1.0 - (X + K3)) - B * (X + K3));
  Result := X + (K1 + 2.0 * K2 + 2.0 * K3 + K4) / 6.0;
end;

function StepLecturerRK4(const P: THHParams; const Phi: Double;
  const S: THHState; const Jin, Dt: Double): THHState;
var
  Rt: THHRates;
  Tmp: THHState;
```

```
  K1, K2, K3, K4: Double;

  function DV(const V: Double): Double;
  var
    JNa, JK, JL: Double;
  begin
    Tmp.V := V;
    IonicCurrents(P, Tmp, JNa, JK, JL);
    Result := (Jin - (JNa + JK + JL)) / P.Cm;
  end;

begin
  // Reproduces the week-9 source code: alpha/beta are evaluated once at
  // v(t); n, m, h are advanced by independent scalar RK4 steps; then v is
  // advanced by a scalar RK4 step with the NEW gate values held constant.
  Rt := ComputeRates(S.V, Phi);
  Result.N := GateRK4(S.N, Rt.An, Rt.Bn, Dt);
  Result.M := GateRK4(S.M, Rt.Am, Rt.Bm, Dt);
  Result.H := GateRK4(S.H, Rt.Ah, Rt.Bh, Dt);
  Tmp := Result;
  K1 := Dt * DV(S.V);
  K2 := Dt * DV(S.V + 0.5 * K1);
  K3 := Dt * DV(S.V + 0.5 * K2);
  K4 := Dt * DV(S.V + K3);
  Result.V := S.V + (K1 + 2.0 * K2 + 2.0 * K3 + K4) / 6.0;
end;
```

c. Alignment of the program output to the membrane voltage

The integrator works with the displacement v of the lecturer's convention, in which rest is $v = 0$ and depolarization is negative. The physiological membrane voltage is obtained by reflecting the sign and adding the resting potential. If V denotes the modern (depolarization-positive) displacement, the two conventions are related by $V = -v$, and the absolute membrane voltage is the resting potential plus this displacement:

$$V_m = V_{rest} + V = V_{rest} - v \qquad (2.1)$$

$$V_m = -70 - v = -(v + 70)\ \text{mV}, \qquad v = 0 \Rightarrow V_m = -70\ \text{mV}, \qquad v = -105 \Rightarrow V_m = +35\ \text{mV} \qquad (2.2)$$

The same reflection explains the stimulus sign, a depolarizing (inward) current must make v more negative, so it enters Equation (1.2) as a negative J_{in}. `AbsoluteVm` implements Equation (2.2), it is the only place where the alignment is made, and every chart, table and CSV export uses its result, while the reversal potentials remain in the lecturer's convention ($v_{Na} = -115$ mV corresponds to $V_m = +45$ mV and $v_K = +12$ mV to $V_m = -82$ mV).

```
function AbsoluteVm(const V: Double): Double;
begin
  // Alignment of the HH displacement v to the absolute membrane voltage
  // Vm = -(v + 70) = V_REST_ABS - v
  Result := V_REST_ABS - V;
end;
```

d. Stimulus protocols, APG run and spike detection

`StimulusAt` returns $J_{in}(t)$ for the three protocols of tab 1: a step from t_{on} to t_{off}, the lecturer's staircase (-5, -10 and -20 µA/cm² on [0,25), [25,38) and [38,50) ms) and the paired pulse used for refractoriness. `AdvanceState` selects the integrator and `CheckStepArguments` rejects non-positive or excessive step counts. `SimulateAPG` starts from the resting steady state, stores t, J_{in}, v, V_m, the three ionic currents, the gates and the conductances $g_{Na} = \bar{g}_{Na} m^3 h$ and $g_K = \bar{g}_K n^4$ at every step, and feeds V_m to the spike detector. An action potential is defined by an upward crossing of $V_m = 0$ mV, `TSpikeDetector.Feed` records the crossing time, follows the peak until the downward crossing and then stores the spike, and `TSpikeDetector.Finish` also counts a spike that is still above 0 mV when the run ends.

```
procedure AppendValue(var A: TDoubleArray; const Value: Double);
var
  L: Integer;
begin
  L := Length(A);
  SetLength(A, L + 1);
  A[L] := Value;
end;

procedure TSpikeDetector.Reset;
begin
  Above := False;
  PeakV := -1.0E30;
  PeakT := 0.0;
  CrossT := 0.0;
  Count := 0;
  SetLength(CrossTimes, 0);
  SetLength(PeakTimes, 0);
  SetLength(PeakValues, 0);
end;

procedure TSpikeDetector.Feed(const T, Vm: Double);
begin
  if not Above then
  begin
    if Vm >= SPIKE_LEVEL then
    begin
      Above := True;
      CrossT := T;
      PeakV := Vm;
      PeakT := T;
    end;
  end
  else
  begin
    if Vm > PeakV then
    begin
      PeakV := Vm;
      PeakT := T;
    end;
```

```
    if Vm < SPIKE_LEVEL then
    begin
      Above := False;
      AppendValue(CrossTimes, CrossT);
      AppendValue(PeakTimes, PeakT);
      AppendValue(PeakValues, PeakV);
      Inc(Count);
    end;
  end;
end;

procedure TSpikeDetector.Finish;
begin
  // A spike still above the detection level when the run ends is counted
  if Above then
  begin
    AppendValue(CrossTimes, CrossT);
    AppendValue(PeakTimes, PeakT);
    AppendValue(PeakValues, PeakV);
    Inc(Count);
    Above := False;
  end;
end;

function DefaultStimulus: TStimulus;
begin
  Result.Kind := skStep;
  Result.JBase := 0.0;
  Result.JLevel := -10.0;
  Result.TOn := 1.0;
  Result.TOff := 61.0;
  Result.PulseWidth := 0.5;
  Result.Interval := 15.0;
  Result.JLevel2 := -40.0;
end;

function StimulusAt(const Stim: TStimulus; const T: Double): Double;
begin
  case Stim.Kind of
    skStep:
      if (T >= Stim.TOn) and (T < Stim.TOff) then
        Result := Stim.JLevel
      else
        Result := Stim.JBase;
    skLecturerStaircase:
      // identical to the lecturer's week-9 source code
      if (T >= 0.0) and (T < 25.0) then
        Result := -5.0
      else if (T >= 25.0) and (T < 38.0) then
        Result := -10.0
      else if (T >= 38.0) and (T < 50.0) then
        Result := -20.0
      else
        Result := 0.0;
    skPairedPulse:
      if (T >= Stim.TOn) and (T < Stim.TOn + Stim.PulseWidth) then
        Result := Stim.JLevel
```

```
      else if (T >= Stim.TOn + Stim.Interval) and
              (T < Stim.TOn + Stim.Interval + Stim.PulseWidth) then
        Result := Stim.JLevel2
      else
        Result := Stim.JBase;
  else
    Result := 0.0;
  end;
end;

function AdvanceState(const P: THHParams; const Phi: Double;
  const Stim: TStimulus; const S: THHState; const T, Dt: Double;
  const Method: TIntegrator): THHState;
begin
  if Method = imCoupledRK4 then
    Result := StepCoupledRK4(P, Phi, S, StimulusAt(Stim, T),
      StimulusAt(Stim, T + 0.5 * Dt), StimulusAt(Stim, T + Dt), Dt)
  else
    Result := StepLecturerRK4(P, Phi, S, StimulusAt(Stim, T), Dt);
end;

procedure CheckStepArguments(const Dt, TEnd: Double);
begin
  if (Dt <= 0.0) or IsNan(Dt) then
    raise EArgumentException.Create('Integration step dt must be positive.
');
  if (TEnd <= 0.0) or IsNan(TEnd) then
    raise EArgumentException.Create('Simulation time must be positive.');
  if TEnd / Dt > 5.0E6 then
    raise EArgumentException.Create(
      'Too many integration steps (TEnd/dt > 5e6). Increase dt or reduce T
End.');
end;

procedure SimulateAPG(const P: THHParams; const Stim: TStimulus;
  const Dt, TEnd: Double; const Method: TIntegrator; out R: TAPGResult);
var
  Phi, T: Double;
  S: THHState;
  I, NSteps: Integer;
  JNa, JK, JL: Double;
  Det: TSpikeDetector;

  procedure Store(const Idx: Integer; const Time, Jin: Double);
  begin
    IonicCurrents(P, S, JNa, JK, JL);
    R.T[Idx] := Time;
    R.V[Idx] := S.V;
    R.Vm[Idx] := AbsoluteVm(S.V);
    R.Jin[Idx] := Jin;
    R.JNa[Idx] := JNa;
    R.JK[Idx] := JK;
    R.JL[Idx] := JL;
    R.M[Idx] := S.M;
    R.H[Idx] := S.H;
    R.N[Idx] := S.N;
    R.GNa[Idx] := P.GNa0 * S.M * S.M * S.M * S.H;
```

```
    R.GK[Idx] := P.GK0 * Sqr(Sqr(S.N));
  end;

begin
  CheckStepArguments(Dt, TEnd);
  Phi := TemperatureFactor(P);
  NSteps := Round(TEnd / Dt);
  R.Count := NSteps + 1;
  SetLength(R.T, R.Count);
  SetLength(R.Vm, R.Count);
  SetLength(R.V, R.Count);
  SetLength(R.Jin, R.Count);
  SetLength(R.JNa, R.Count);
  SetLength(R.JK, R.Count);
  SetLength(R.JL, R.Count);
  SetLength(R.M, R.Count);
  SetLength(R.H, R.Count);
  SetLength(R.N, R.Count);
  SetLength(R.GNa, R.Count);
  SetLength(R.GK, R.Count);

  // Initial condition: membrane at rest, gates at their steady state
  S := SteadyState(0.0, Phi);
  Det.Reset;
  Store(0, 0.0, StimulusAt(Stim, 0.0));
  Det.Feed(0.0, R.Vm[0]);
  for I := 1 to NSteps do
  begin
    T := (I - 1) * Dt;
    S := AdvanceState(P, Phi, Stim, S, T, Dt, Method);
    Store(I, I * Dt, StimulusAt(Stim, I * Dt));
    Det.Feed(I * Dt, R.Vm[I]);
  end;
  Det.Finish;
  R.SpikeCross := Copy(Det.CrossTimes);
  R.SpikeTimes := Copy(Det.PeakTimes);
  R.SpikePeaks := Copy(Det.PeakValues);
end;

procedure SimulateSpikesOnly(const P: THHParams; const Stim: TStimulus;
  const Dt, TEnd: Double; const Method: TIntegrator;
  var Det: TSpikeDetector);
var
  Phi: Double;
  S: THHState;
  I, NSteps: Integer;
begin
  CheckStepArguments(Dt, TEnd);
  Phi := TemperatureFactor(P);
  NSteps := Round(TEnd / Dt);
  S := SteadyState(0.0, Phi);
  Det.Reset;
  Det.Feed(0.0, AbsoluteVm(S.V));
  for I := 1 to NSteps do
  begin
    S := AdvanceState(P, Phi, Stim, S, (I - 1) * Dt, Dt, Method);
    Det.Feed(I * Dt, AbsoluteVm(S.V));
```

```
  end;
  Det.Finish;
end;
```

On the form, `btnRunAPGClick` reads the protocol and the parameters, validates them, runs `SimulateAPG` and calls the plotting and report routines. `PlotAPG` draws V_m with J_{in} on a right axis, the three ionic currents (J_L on its own right axis because it is two orders of magnitude smaller than J_{Na} and J_K), the gates and the conductances, the four time axes are synchronized when zooming. `PlotPhase` draws the trajectories of m, h and n against V_m as dots at a constant time stride over the steady-state curves $x_\infty(V_m)$, together with $\tau_x(V_m)$ on a logarithmic axis, which makes the lag of each gate behind its steady state visible. `ReportAPG` lists the spike times, peaks and inter-spike intervals in the result memo.

```
procedure TForm1.btnRunAPGClick(Sender: TObject);
var
  LParams: THHParams;
  LStim: TStimulus;
  LDt, LTEnd: Double;
begin
  try
    LParams := ReadModelParams;
    LStim := DefaultStimulus;
    case rgProtocol.ItemIndex of
      1:
        LStim.Kind := skLecturerStaircase;
      2:
        begin
          LStim.Kind := skPairedPulse;
          LStim.JBase := ParamValue(vleAPG, K_JBASE);
          LStim.TOn := ParamValue(vleAPG, K_TON);
          LStim.JLevel := ParamValue(vleAPG, K_JP1);
          LStim.JLevel2 := ParamValue(vleAPG, K_JP2);
          LStim.PulseWidth := ParamValue(vleAPG, K_PW);
          LStim.Interval := ParamValue(vleAPG, K_PINT);
          if LStim.PulseWidth <= 0.0 then
            raise EConvertError.Create('Pulse width must be positive.');
          if LStim.Interval < LStim.PulseWidth then
            raise EConvertError.Create(
              'Pulse interval must not be shorter than the pulse width.');
        end;
    else
      LStim.Kind := skStep;
      LStim.JBase := ParamValue(vleAPG, K_JBASE);
      LStim.JLevel := ParamValue(vleAPG, K_JLEVEL);
      LStim.TOn := ParamValue(vleAPG, K_TON);
      LStim.TOff := ParamValue(vleAPG, K_TOFF);
      if LStim.TOff <= LStim.TOn then
        raise EConvertError.Create('t_off must be larger than t_on.');
    end;
    LDt := ParamValue(vleAPG, K_DT);
    LTEnd := ParamValue(vleAPG, K_TEND);

    Screen.Cursor := crHourGlass;
    try
```

```
      SimulateAPG(LParams, LStim, LDt, LTEnd, SelectedMethod, FAPG);
    finally
      Screen.Cursor := crDefault;
    end;
    FAPGValid := True;
    PlotAPG;
    PlotPhase(LParams);
    ReportAPG(LStim, LDt, LTEnd);
    SetStatus(Format('APG finished: %d samples, %d action potentials.',
      [FAPG.Count, Length(FAPG.SpikeTimes)]));
  except
    on E: Exception do
      ShowError(E.Message);
  end;
end;

procedure TForm1.PlotAPG;
var
  LVm, LJin, LNa, LK, LL, LM, LH, LN, LgNa, LgK: TFastLineSeries;
  LIdx, LStride: Integer;
begin
  ClearChart(chtVm);
  ClearChart(chtCurrents);
  ClearChart(chtGates);
  ClearChart(chtConductance);
  LVm := AddLine(chtVm, 'Vm', C_BLACK);
  LJin := AddLine(chtVm, 'Jin (right axis)', C_STIM, True);
  LNa := AddLine(chtCurrents, 'JNa', C_VERMILLION);
  LK := AddLine(chtCurrents, 'JK', C_BLUE);
  // JL (about +/-25 uA/cm2) would be invisible next to JNa/JK (about
  // +/-800 uA/cm2), so it gets its own right axis; Jin is already in chtV
m.
  LL := AddLine(chtCurrents, 'JL (right axis)', C_GREEN, True);
  LM := AddLine(chtGates, 'm (Na activation)', C_VERMILLION);
  LH := AddLine(chtGates, 'h (Na inactivation)', C_ORANGE);
  LN := AddLine(chtGates, 'n (K activation)', C_BLUE);
  LgNa := AddLine(chtConductance, 'gNa = gNa0 m^3 h', C_VERMILLION);
  LgK := AddLine(chtConductance, 'gK = gK0 n^4', C_BLUE);

  // at most about 8000 points per series keeps TeeChart responsive
  LStride := Max(1, FAPG.Count div 8000);
  LIdx := 0;
  while LIdx < FAPG.Count do
  begin
    LVm.AddXY(FAPG.T[LIdx], FAPG.Vm[LIdx]);
    LJin.AddXY(FAPG.T[LIdx], FAPG.Jin[LIdx]);
    LNa.AddXY(FAPG.T[LIdx], FAPG.JNa[LIdx]);
    LK.AddXY(FAPG.T[LIdx], FAPG.JK[LIdx]);
    LL.AddXY(FAPG.T[LIdx], FAPG.JL[LIdx]);
    LM.AddXY(FAPG.T[LIdx], FAPG.M[LIdx]);
    LH.AddXY(FAPG.T[LIdx], FAPG.H[LIdx]);
    LN.AddXY(FAPG.T[LIdx], FAPG.N[LIdx]);
    LgNa.AddXY(FAPG.T[LIdx], FAPG.GNa[LIdx]);
    LgK.AddXY(FAPG.T[LIdx], FAPG.GK[LIdx]);
    Inc(LIdx, LStride);
  end;
  FinishChart(chtVm);
```

```
  FinishChart(chtCurrents);
  FinishChart(chtGates);
  FinishChart(chtConductance);
end;

procedure TForm1.PlotPhase(const AParams: THHParams);
const
  C_VM_FROM = -100.0;   // range of the steady-state curves (mV)
  C_VM_TO = 50.0;
  C_VM_STEP = 0.5;
var
  LTrajM, LTrajH, LTrajN: TPointSeries;
  LInfM, LInfH, LInfN, LTauM, LTauH, LTauN: TFastLineSeries;
  LIdx, LStride, LCount: Integer;
  LPhi, LVm, LV: Double;
  LRates: THHRates;
begin
  // Phase plots in the style of the senior reference (gate vs Vm), with t
he
  // steady-state curve x_inf(Vm) for comparison: the loop is the delay of
  // each gate behind x_inf, whose size is set by tau_x(Vm). Trajectories
are
  // dots taken at a constant time stride, so the dot spacing shows how fa
st
  // the state moves (dense = slow recovery, sparse = AP upstroke).
  ClearChart(chtPhaseM);
  ClearChart(chtPhaseH);
  ClearChart(chtPhaseN);
  ClearChart(chtTau);
  LTrajM := AddPoints(chtPhaseM, 'm trajectory (simulation)', C_VERMILLION
);
  LInfM := AddLine(chtPhaseM, 'm_inf(Vm) steady state', C_STIM);
  LTrajH := AddPoints(chtPhaseH, 'h trajectory (simulation)', C_ORANGE);
  LInfH := AddLine(chtPhaseH, 'h_inf(Vm) steady state', C_STIM);
  LTrajN := AddPoints(chtPhaseN, 'n trajectory (simulation)', C_BLUE);
  LInfN := AddLine(chtPhaseN, 'n_inf(Vm) steady state', C_STIM);
  LTauM := AddLine(chtTau, 'tau_m', C_VERMILLION);
  LTauH := AddLine(chtTau, 'tau_h', C_ORANGE);
  LTauN := AddLine(chtTau, 'tau_n', C_BLUE);

  LStride := Max(1, FAPG.Count div 4000);
  LIdx := 0;
  while LIdx < FAPG.Count do
  begin
    LTrajM.AddXY(FAPG.Vm[LIdx], FAPG.M[LIdx]);
    LTrajH.AddXY(FAPG.Vm[LIdx], FAPG.H[LIdx]);
    LTrajN.AddXY(FAPG.Vm[LIdx], FAPG.N[LIdx]);
    Inc(LIdx, LStride);
  end;

  LPhi := TemperatureFactor(AParams);
  LCount := Round((C_VM_TO - C_VM_FROM) / C_VM_STEP);
  for LIdx := 0 to LCount do
  begin
    LVm := C_VM_FROM + LIdx * C_VM_STEP;
    LV := V_REST_ABS - LVm;            // HH displacement: v = -(Vm + 70)
    LRates := ComputeRates(LV, LPhi);
```

```
    LInfM.AddXY(LVm, LRates.Am / (LRates.Am + LRates.Bm));
    LInfH.AddXY(LVm, LRates.Ah / (LRates.Ah + LRates.Bh));
    LInfN.AddXY(LVm, LRates.An / (LRates.An + LRates.Bn));
    LTauM.AddXY(LVm, 1.0 / (LRates.Am + LRates.Bm));
    LTauH.AddXY(LVm, 1.0 / (LRates.Ah + LRates.Bh));
    LTauN.AddXY(LVm, 1.0 / (LRates.An + LRates.Bn));
  end;
  FinishChart(chtPhaseM);
  FinishChart(chtPhaseH);
  FinishChart(chtPhaseN);
  FinishChart(chtTau);
end;

procedure TForm1.ReportAPG(const AStim: TStimulus; const ADt, ATEnd: Doubl
e);
var
  LLines: TStrings;
  LIdx, LCount: Integer;
  LSpan: Double;
begin
  LLines := memAPG.Lines;
  LLines.BeginUpdate;
  try
    LLines.Clear;
    case AStim.Kind of
      skStep:
        LLines.Add(Format('Protocol : step %s uA/cm2, %s-%s ms (baseline %
s)',
          [FloatText(AStim.JLevel, 2), FloatText(AStim.TOn, 2),
           FloatText(AStim.TOff, 2), FloatText(AStim.JBase, 2)]));
      skLecturerStaircase:
        LLines.Add('Protocol : lecturer staircase -5/-10/-20/0 uA/cm2 ' +
          '(0-25/25-38/38-50/>50 ms)');
      skPairedPulse:
        LLines.Add(Format('Protocol : paired pulse J1=%s, J2=%s uA/cm2, '
+
          'width %s ms, interval %s ms',
          [FloatText(AStim.JLevel, 2), FloatText(AStim.JLevel2, 2),
           FloatText(AStim.PulseWidth, 2), FloatText(AStim.Interval, 2)]))
;
    end;
    if SelectedMethod = imCoupledRK4 then
      LLines.Add('Method   : coupled RK4 on [v, m, h, n]')
    else
      LLines.Add('Method   : lecturer sequential RK4');
    LLines.Add(Format('dt = %s ms, T = %s ms, rest Vm = %s mV',
      [FloatText(ADt, 4), FloatText(ATEnd, 1), FloatText(FAPG.Vm[0], 2)]))
;
    LCount := Length(FAPG.SpikeTimes);
    LLines.Add(Format('Action potentials (Vm crosses 0 mV): %d', [LCount])
);
    if LCount > 0 then
    begin
      LLines.Add(' #   t_cross(ms)  t_peak(ms)  Vpeak(mV)  ISI(ms)');
      for LIdx := 0 to LCount - 1 do
        if LIdx = 0 then
          LLines.Add(Format('%2d   %9s   %9s   %8s      -',
```

```
            [LIdx + 1, FloatText(FAPG.SpikeCross[LIdx], 2),
             FloatText(FAPG.SpikeTimes[LIdx], 2),
             FloatText(FAPG.SpikePeaks[LIdx], 2)]))
        else
          LLines.Add(Format('%2d   %9s   %9s   %8s  %7s',
            [LIdx + 1, FloatText(FAPG.SpikeCross[LIdx], 2),
             FloatText(FAPG.SpikeTimes[LIdx], 2),
             FloatText(FAPG.SpikePeaks[LIdx], 2),
             FloatText(FAPG.SpikeTimes[LIdx] - FAPG.SpikeTimes[LIdx - 1],
2)]));
      if LCount > 1 then
      begin
        LSpan := FAPG.SpikeTimes[LCount - 1] - FAPG.SpikeTimes[0];
        LLines.Add(Format('Mean ISI = %s ms  ->  mean frequency = %s Hz',
          [FloatText(LSpan / (LCount - 1), 3),
           FloatText(1000.0 * (LCount - 1) / LSpan, 2)]));
      end;
    end;
    if AStim.Kind = skPairedPulse then
    begin
      if LCount >= 2 then
        LLines.Add('Second pulse produced an action potential (outside the
' +
          'refractory period).')
      else
        LLines.Add('Second pulse did NOT produce an action potential: the
' +
          'membrane is refractory at this interval/amplitude.');
    end;
  finally
    LLines.EndUpdate;
  end;
end;
```

e. Refractory period

The refractory period is measured with the paired-pulse protocol of Section 1.6. `PairedStimulus` builds a conditioning pulse J_1 of width w at $t_{on} = 1$ ms and a test pulse J_2 at $t_{on} + \Delta T$. `SecondSpikeOccurs` integrates the model and reports whether a spike crosses 0 mV after the onset of the test pulse. `SinglePulseThreshold` determines the rested threshold $J_{th,0}$ and `SecondPulseThreshold` the test threshold $J_{th}(\Delta T)$ by bisection on $[0, |J_2|_{max}]$. Bisection is valid because the response is monotone in the amplitude: if an amplitude elicits a spike, every larger one does as well. After 16 halvings the threshold is bracketed to $|J_2|_{max}/2^{16}$, about 0.008 µA/cm² for $|J_2|_{max} = 500$ µA/cm². The definitions used for the two phases of refractoriness are:

$$J_{th}(\Delta T) = \min\{|J_2|:\ \text{a second AP occurs}\},\quad |J_2| > |J_2|_{max},\qquad \begin{matrix}\text{absolute: } J_{th}(\Delta T)\\ \text{relative: } J_{th,0} < J_{th}(\Delta T)\end{matrix} \qquad (2.3)$$

and the relative refractory period is taken to end when $J_{th}(\Delta T) \le 1.05\, J_{th,0}$. On the form, `btnRunRefClick` sweeps the interval, plots $J_{th}(\Delta T)$ with the rested threshold as a reference and

the peak of the second action potential on a right axis, and derives both phase durations; `RunRefExample` simulates the paired-pulse response for the interval selected with the track bar.

```
function PairedStimulus(const J1, J2, Width, Interval: Double): TStimulus;
begin
  Result := DefaultStimulus;
  Result.Kind := skPairedPulse;
  Result.JBase := 0.0;
  Result.JLevel := J1;
  Result.JLevel2 := J2;
  Result.TOn := PAIRED_T_ON;
  Result.PulseWidth := Width;
  Result.Interval := Interval;
end;

function SecondSpikeOccurs(const P: THHParams; const J1, J2, Width,
  Interval, Dt: Double; const Method: TIntegrator;
  out SecondPeak: Double): Boolean;
var
  Det: TSpikeDetector;
  TTest: Double;
  I: Integer;
begin
  // The test response is a spike whose upward 0 mV crossing occurs after
  // the onset of the second (test) pulse.
  TTest := PAIRED_T_ON + Interval;
  SimulateSpikesOnly(P, PairedStimulus(J1, J2, Width, Interval), Dt,
    TTest + PAIRED_T_AFTER, Method, Det);
  Result := False;
  SecondPeak := NaN;
  I := 0;
  while (not Result) and (I < Det.Count) do
  begin
    if Det.CrossTimes[I] >= TTest then
    begin
      Result := True;
      SecondPeak := Det.PeakValues[I];
    end;
    Inc(I);
  end;
end;

function SinglePulseThreshold(const P: THHParams; const Sign, Width,
  JMaxAbs, Dt: Double; const Method: TIntegrator): Double;
var
  Lo, Hi, Mid: Double;
  K: Integer;
  Det: TSpikeDetector;
  Stim: TStimulus;

  function Fires(const A: Double): Boolean;
  begin
    Stim := PairedStimulus(Sign * A, 0.0, Width, 1.0E6);
    SimulateSpikesOnly(P, Stim, Dt, PAIRED_T_ON + PAIRED_T_AFTER, Method,
Det);
    Result := Det.Count > 0;
  end;
```

```
begin
  // Smallest |J| of a single pulse that elicits an action potential
  if not Fires(JMaxAbs) then
    Exit(NaN);
  Lo := 0.0;
  Hi := JMaxAbs;
  for K := 1 to BISECTION_STEPS do
  begin
    Mid := 0.5 * (Lo + Hi);
    if Fires(Mid) then
      Hi := Mid
    else
      Lo := Mid;
  end;
  Result := Hi;
end;

function SecondPulseThreshold(const P: THHParams; const J1, Width,
  Interval, JMaxAbs, Dt: Double; const Method: TIntegrator): Double;
var
  Lo, Hi, Mid, Pk, Sign: Double;
  K: Integer;
begin
  // Minimum |J2| of the test pulse that produces a second action potentia
l.
  // NaN means no response up to JMaxAbs: absolute refractory period.
  if J1 < 0.0 then
    Sign := -1.0
  else
    Sign := 1.0;
  if not SecondSpikeOccurs(P, J1, Sign * JMaxAbs, Width, Interval, Dt,
    Method, Pk) then
    Exit(NaN);
  Lo := 0.0;
  Hi := JMaxAbs;
  for K := 1 to BISECTION_STEPS do
  begin
    Mid := 0.5 * (Lo + Hi);
    if SecondSpikeOccurs(P, J1, Sign * Mid, Width, Interval, Dt, Method, P
k) then
      Hi := Mid
    else
      Lo := Mid;
  end;
  Result := Hi;
end;

procedure TForm1.btnRunRefClick(Sender: TObject);
var
  LParams: THHParams;
  LJ1, LWidth, LI0, LI1, LStep, LJMax, LDt, LPeak, LSign: Double;
  LCount, LIdx, LAbsIdx, LRelIdx: Integer;
  LThr, LBase, LPk: TLineSeries;
  LLines: TStrings;
begin
  if FBusy then
```

```
    Exit;
  FBusy := True;
  try
    LParams := ReadModelParams;
    LJ1 := ParamValue(vleRef, K_RJ1);
    LWidth := ParamValue(vleRef, K_RPW);
    LI0 := ParamValue(vleRef, K_RI0);
    LI1 := ParamValue(vleRef, K_RI1);
    LStep := ParamValue(vleRef, K_RIS);
    LJMax := Abs(ParamValue(vleRef, K_RJMAX));
    LDt := ParamValue(vleRef, K_DT);
    if (LWidth <= 0.0) or (LStep <= 0.0) or (LI1 < LI0) or (LI0 < LWidth)
then
      raise EConvertError.Create('Require width > 0, step > 0, ' +
        'interval start >= width and interval end >= interval start.');
    if LJ1 < 0.0 then
      LSign := -1.0
    else
      LSign := 1.0;
    LCount := Floor((LI1 - LI0) / LStep + 1.0E-9) + 1;
    SetLength(FRefInterval, LCount);
    SetLength(FRefThreshold, LCount);
    SetLength(FRefPeak, LCount);

    Screen.Cursor := crHourGlass;
    try
      FRefBaseline := SinglePulseThreshold(LParams, LSign, LWidth, LJMax,
LDt,
        SelectedMethod);
      for LIdx := 0 to LCount - 1 do
      begin
        FRefInterval[LIdx] := LI0 + LIdx * LStep;
        FRefThreshold[LIdx] := SecondPulseThreshold(LParams, LJ1, LWidth,
          FRefInterval[LIdx], LJMax, LDt, SelectedMethod);
        if SecondSpikeOccurs(LParams, LJ1, LJ1, LWidth, FRefInterval[LIdx]
,
          LDt, SelectedMethod, LPeak) then
          FRefPeak[LIdx] := LPeak
        else
          FRefPeak[LIdx] := NaN;
        SetStatus(Format('Refractory sweep %d / %d', [LIdx + 1, LCount]));
        Application.ProcessMessages;
      end;
    finally
      Screen.Cursor := crDefault;
    end;

    ClearChart(chtRefThreshold);
    LThr := AddMarkedLine(chtRefThreshold, 'Threshold of test pulse |J2|',
      C_BLUE);
    LBase := AddMarkedLine(chtRefThreshold, 'Rested single-pulse threshold
',
      C_STIM);
    LBase.Pointer.Visible := False;
    LPk := AddMarkedLine(chtRefThreshold,
      'Peak of 2nd AP for J2 = J1 (right axis)', C_VERMILLION, True);
    for LIdx := 0 to LCount - 1 do
```

```
    begin
      if IsNan(FRefThreshold[LIdx]) then
        LThr.AddNullXY(FRefInterval[LIdx], 0)
      else
        LThr.AddXY(FRefInterval[LIdx], FRefThreshold[LIdx]);
      if IsNan(FRefPeak[LIdx]) then
        LPk.AddNullXY(FRefInterval[LIdx], 0)
      else
        LPk.AddXY(FRefInterval[LIdx], FRefPeak[LIdx]);
    end;
    if not IsNan(FRefBaseline) then
    begin
      LBase.AddXY(FRefInterval[0], FRefBaseline);
      LBase.AddXY(FRefInterval[LCount - 1], FRefBaseline);
    end;
    FinishChart(chtRefThreshold);

    // absolute refractory: no response up to JMax;
    // relative refractory ends when threshold <= 105 % of the rested valu
e
    LAbsIdx := -1;
    for LIdx := 0 to LCount - 1 do
      if IsNan(FRefThreshold[LIdx]) then
        LAbsIdx := LIdx;
    LRelIdx := -1;
    LIdx := LAbsIdx + 1;
    while (LRelIdx < 0) and (LIdx < LCount) do
    begin
      if (not IsNan(FRefThreshold[LIdx])) and (not IsNan(FRefBaseline)) an
d
         (FRefThreshold[LIdx] <= 1.05 * FRefBaseline) then
        LRelIdx := LIdx;
      Inc(LIdx);
    end;

    LLines := memRef.Lines;
    LLines.BeginUpdate;
    try
      LLines.Clear;
      LLines.Add(Format('Conditioning pulse J1 = %s uA/cm2, width = %s ms'
,
        [FloatText(LJ1, 2), FloatText(LWidth, 3)]));
      LLines.Add(Format('Rested single-pulse threshold = %s uA/cm2',
        [FloatText(FRefBaseline, 3)]));
      if LAbsIdx >= 0 then
        LLines.Add(Format('Absolute refractory: no AP up to |J2| = %s for
' +
          'intervals <= %s ms', [FloatText(LJMax, 0),
          FloatText(FRefInterval[LAbsIdx], 2)]))
      else
        LLines.Add('No absolute refractory interval inside the sweep range
.');
      if LRelIdx >= 0 then
        LLines.Add(Format('Relative refractory ends at about %s ms ' +
          '(threshold <= 1.05 x rested)', [FloatText(FRefInterval[LRelIdx]
, 2)]))
      else
```

```
        LLines.Add('Threshold did not recover to 1.05 x rested in the rang
e.');
      LLines.Add('');
      LLines.Add(' interval(ms)  |J2|thr(uA/cm2)  thr/rested  peak2(J2=J1)
');
      for LIdx := 0 to LCount - 1 do
        if IsNan(FRefThreshold[LIdx]) or IsNan(FRefBaseline) then
          LLines.Add(Format('%10s   %12s   %9s   %8s',
            [FloatText(FRefInterval[LIdx], 2), FloatText(FRefThreshold[LId
x], 3),
             'n/a', FloatText(FRefPeak[LIdx], 2)]))
        else
          LLines.Add(Format('%10s   %12s   %9s   %8s',
            [FloatText(FRefInterval[LIdx], 2), FloatText(FRefThreshold[LId
x], 3),
             FloatText(FRefThreshold[LIdx] / FRefBaseline, 3),
             FloatText(FRefPeak[LIdx], 2)]));
    finally
      LLines.EndUpdate;
    end;
    SetStatus('Refractory sweep finished. Move the track bar to inspect '
+
      'single intervals.');
  except
    on E: Exception do
      ShowError(E.Message);
  end;
  FBusy := False;
  RunRefExample;
end;

procedure TForm1.RunRefExample;
var
  LParams: THHParams;
  LStim: TStimulus;
  LResult: TAPGResult;
  LJ1, LWidth, LInterval, LJMax, LDt, LThr, LSign, LTEnd: Double;

  procedure PlotTrace(ASlot: Integer; const ATitle: string);
  var
    LSeries: TFastLineSeries;
    LIdx: Integer;
  begin
    // the three series are permanent; only their points and titles change
    LSeries := FRefExSeries[ASlot];
    LSeries.BeginUpdate;
    try
      LSeries.Clear;
      for LIdx := 0 to LResult.Count - 1 do
        LSeries.AddXY(LResult.T[LIdx], LResult.Vm[LIdx]);
    finally
      LSeries.EndUpdate;
    end;
    LSeries.Title := ATitle;
    LSeries.Active := True;
  end;
```

```
begin
  if FBusy then
    Exit;
  FBusy := True;
  try
    LParams := ReadModelParams;
    LJ1 := ParamValue(vleRef, K_RJ1);
    LWidth := ParamValue(vleRef, K_RPW);
    LInterval := RefExampleInterval;
    LJMax := Abs(ParamValue(vleRef, K_RJMAX));
    LDt := ParamValue(vleRef, K_DT);
    if (LWidth <= 0.0) or (LInterval < LWidth) then
      raise EConvertError.Create('Require width > 0 and interval >= width.
');
    if LJ1 < 0.0 then
      LSign := -1.0
    else
      LSign := 1.0;
    LTEnd := 1.0 + LInterval + 20.0;

    LStim := DefaultStimulus;
    LStim.Kind := skPairedPulse;
    LStim.TOn := 1.0;
    LStim.JLevel := LJ1;
    LStim.PulseWidth := LWidth;
    LStim.Interval := LInterval;

    Screen.Cursor := crHourGlass;
    try
      LStim.JLevel2 := 0.0;
      SimulateAPG(LParams, LStim, LDt, LTEnd, SelectedMethod, LResult);
      PlotTrace(0, 'Conditioning pulse only');

      LStim.JLevel2 := LJ1;
      SimulateAPG(LParams, LStim, LDt, LTEnd, SelectedMethod, LResult);
      PlotTrace(1, Format('Test pulse J2 = J1 = %s', [FloatText(LJ1, 1)]))
;

      LThr := SecondPulseThreshold(LParams, LJ1, LWidth, LInterval, LJMax,
LDt,
        SelectedMethod);
      if not IsNan(LThr) then
      begin
        LStim.JLevel2 := LSign * 1.05 * LThr;
        SimulateAPG(LParams, LStim, LDt, LTEnd, SelectedMethod, LResult);
        PlotTrace(2, Format('Test pulse J2 = 1.05 x threshold = %s',
          [FloatText(LStim.JLevel2, 1)]));
        SetStatus(Format('Interval %s ms: test-pulse threshold = %s uA/cm2
',
          [FloatText(LInterval, 2), FloatText(LThr, 3)]));
      end
      else
      begin
        // absolute refractory: keep the legend entry, hide the series
        FRefExSeries[2].Clear;
        FRefExSeries[2].Title := Format('No AP up to |J2| = %s ' +
          '(absolute refractory)', [FloatText(LJMax, 0)]);
```

```
        FRefExSeries[2].Active := False;
        SetStatus(Format('Interval %s ms lies in the absolute refractory '
+
          'period (no AP up to |J2| = %s).', [FloatText(LInterval, 2),
          FloatText(LJMax, 0)]));
      end;
    finally
      Screen.Cursor := crDefault;
    end;
    FinishChart(chtRefExample);
  except
    on E: Exception do
      ShowError(E.Message);
  end;
  FBusy := False;
end;
```

f. How the Jin level programs the inter-pulse interval

`SteadyFiring` applies a constant J_{in} from $t = 0$, discards the spikes of the initial transient and computes the mean steady inter-spike interval from the peak times $t_{p,i}$, the firing frequency follows from the interval in milliseconds:

$$\mathrm{ISI}_i = t_{p,i} - t_{p,i-1}, \qquad \overline{\mathrm{ISI}} = \frac{1}{N_s} \sum_{t_{p,i-1} \geq t_{dis}} \mathrm{ISI}_i\,, \qquad f = \frac{1000}{\overline{\mathrm{ISI}}}\ \mathrm{Hz} \qquad (2.4)$$

The mechanism can be read from Equation (1.2). Between two spikes the membrane is charged by $J_{in} - J_{ion}$ through C_m, so the time needed to climb back to threshold scales roughly with $C_m \Delta V_{th} / |J_{in} - J_{ion}|$: a larger $|J_{in}|$ shortens the interval, while the refractoriness of Section 1.6 sets a lower bound below which no shortening is possible. `btnRunISIClick` sweeps J_{in}, plots $\overline{\mathrm{ISI}}$ and f against $|J_{in}|$ and draws three example spike trains.

```
procedure SteadyFiring(const P: THHParams; const J, Dt, TEnd,
  TDiscard: Double; const Method: TIntegrator;
  out NSpikes: Integer; out MeanISI: Double);
var
  Stim: TStimulus;
  Det: TSpikeDetector;
  I, NUsed: Integer;
  SumISI: Double;
begin
  // Constant current J switched on at t = 0. The mean inter-spike interva
l is
  // computed from spikes occurring after TDiscard (transient removed).
  Stim := DefaultStimulus;
  Stim.Kind := skStep;
  Stim.JBase := 0.0;
  Stim.JLevel := J;
  Stim.TOn := 0.0;
  Stim.TOff := TEnd + 1.0;
  SimulateSpikesOnly(P, Stim, Dt, TEnd, Method, Det);
```

```
  NSpikes := Det.Count;
  SumISI := 0.0;
  NUsed := 0;
  for I := 1 to Det.Count - 1 do
    if Det.PeakTimes[I - 1] >= TDiscard then
    begin
      SumISI := SumISI + (Det.PeakTimes[I] - Det.PeakTimes[I - 1]);
      Inc(NUsed);
    end;
  if NUsed > 0 then
    MeanISI := SumISI / NUsed
  else
    MeanISI := NaN;
end;

procedure TForm1.btnRunISIClick(Sender: TObject);
var
  LParams: THHParams;
  LJ0, LJ1, LJStep, LTRun, LTDis, LDt, LJ, LISI: Double;
  LCount, LIdx, LSpikes, LRheo, LRep, LBlock, LK: Integer;
  LISISeries, LFreq: TLineSeries;
  LStim: TStimulus;
  LResult: TAPGResult;
  LLines: TStrings;
  LTraceJ: array[0..2] of Double;
  LSeries: TFastLineSeries;
begin
  if FBusy then
    Exit;
  FBusy := True;
  try
    LParams := ReadModelParams;
    LJ0 := ParamValue(vleISI, K_IJ0);
    LJ1 := ParamValue(vleISI, K_IJ1);
    LJStep := Abs(ParamValue(vleISI, K_IJS));
    LTRun := ParamValue(vleISI, K_ITR);
    LTDis := ParamValue(vleISI, K_IDIS);
    LDt := ParamValue(vleISI, K_DT);
    if (LJStep <= 0.0) or (LTRun <= LTDis) or (LTDis < 0.0) then
      raise EConvertError.Create('Require step > 0 and 0 <= discard < run
time.');
    LCount := Floor(Abs(LJ1 - LJ0) / LJStep + 1.0E-9) + 1;
    if LCount > 2000 then
      raise EConvertError.Create('Too many Jin levels (> 2000).');
    SetLength(FISIJin, LCount);
    SetLength(FISIValue, LCount);
    SetLength(FISICount, LCount);

    Screen.Cursor := crHourGlass;
    try
      for LIdx := 0 to LCount - 1 do
      begin
        LJ := LJ0 + Sign(LJ1 - LJ0) * LIdx * LJStep;
        FISIJin[LIdx] := LJ;
        SteadyFiring(LParams, LJ, LDt, LTRun, LTDis, SelectedMethod, LSpik
es,
          LISI);
```

```
        FISICount[LIdx] := LSpikes;
        FISIValue[LIdx] := LISI;
        if (LIdx mod 5) = 0 then
        begin
          SetStatus(Format('Jin sweep %d / %d', [LIdx + 1, LCount]));
          Application.ProcessMessages;
        end;
      end;
    finally
      Screen.Cursor := crDefault;
    end;

    ClearChart(chtISI);
    LISISeries := AddMarkedLine(chtISI, 'Steady inter-pulse interval', C_B
LUE);
    LFreq := AddMarkedLine(chtISI, 'Firing frequency (right axis)',
      C_VERMILLION, True);
    for LIdx := 0 to LCount - 1 do
      if IsNan(FISIValue[LIdx]) then
      begin
        LISISeries.AddNullXY(Abs(FISIJin[LIdx]), 0);
        LFreq.AddNullXY(Abs(FISIJin[LIdx]), 0);
      end
      else
      begin
        LISISeries.AddXY(Abs(FISIJin[LIdx]), FISIValue[LIdx]);
        LFreq.AddXY(Abs(FISIJin[LIdx]), 1000.0 / FISIValue[LIdx]);
      end;
    FinishChart(chtISI);

    // example spike trains, coloured by |Jin| (ordered palette)
    LTraceJ[0] := ParamValue(vleISI, K_ITA);
    LTraceJ[1] := ParamValue(vleISI, K_ITB);
    LTraceJ[2] := ParamValue(vleISI, K_ITC);
    ClearChart(chtISITraces);
    for LK := 0 to 2 do
    begin
      LStim := DefaultStimulus;
      LStim.Kind := skStep;
      LStim.JLevel := LTraceJ[LK];
      LStim.TOn := 0.0;
      LStim.TOff := LTRun + 1.0;
      SimulateAPG(LParams, LStim, LDt, Min(LTRun, 100.0), SelectedMethod,
        LResult);
      LSeries := AddLine(chtISITraces, Format('Jin = %s uA/cm2',
        [FloatText(LTraceJ[LK], 1)]), OrderedLineColor(LK, 3));
      LIdx := 0;
      while LIdx < LResult.Count do
      begin
        LSeries.AddXY(LResult.T[LIdx], LResult.Vm[LIdx]);
        Inc(LIdx, Max(1, LResult.Count div 6000));
      end;
    end;
    FinishChart(chtISITraces);

    // summary: first AP, onset of repetitive firing, depolarization block
    LRheo := -1;
```

```
    LRep := -1;
    LBlock := -1;
    for LIdx := 0 to LCount - 1 do
    begin
      if (LRheo < 0) and (FISICount[LIdx] >= 1) then
        LRheo := LIdx;
      if (LRep < 0) and (not IsNan(FISIValue[LIdx])) then
        LRep := LIdx;
      if (LRep >= 0) and (LBlock < 0) and (LIdx > LRep) and
         IsNan(FISIValue[LIdx]) then
        LBlock := LIdx;
    end;
    LLines := memISI.Lines;
    LLines.BeginUpdate;
    try
      LLines.Clear;
      LLines.Add(Format('Constant Jin applied from t = 0; ISI averaged aft
er %s ms.',
        [FloatText(LTDis, 1)]));
      if LRheo >= 0 then
        LLines.Add(Format('First Jin with at least one AP     : %s uA/cm2'
,
          [FloatText(FISIJin[LRheo], 2)]));
      if LRep >= 0 then
        LLines.Add(Format('Onset of repetitive (tonic) firing : %s uA/cm2'
,
          [FloatText(FISIJin[LRep], 2)]));
      if LBlock >= 0 then
        LLines.Add(Format('Repetitive firing lost (block) at  : %s uA/cm2'
,
          [FloatText(FISIJin[LBlock], 2)]));
      LLines.Add('');
      LLines.Add('  Jin(uA/cm2)  spikes   ISI(ms)   f(Hz)');
      for LIdx := 0 to LCount - 1 do
        if IsNan(FISIValue[LIdx]) then
          LLines.Add(Format('%11s  %6d   %7s   %6s',
            [FloatText(FISIJin[LIdx], 2), FISICount[LIdx], 'n/a', 'n/a']))
        else
          LLines.Add(Format('%11s  %6d   %7s   %6s',
            [FloatText(FISIJin[LIdx], 2), FISICount[LIdx],
             FloatText(FISIValue[LIdx], 3),
             FloatText(1000.0 / FISIValue[LIdx], 2)]));
    finally
      LLines.EndUpdate;
    end;
    SetStatus('Jin sweep finished.');
  except
    on E: Exception do
      ShowError(E.Message);
  end;
  FBusy := False;
end;
```

2.2.3 Problem 2. Derivation of the APP Integration with Central and Forward Differences

Step 1, the partial differential equation. Consider the segment between x and $x + \Delta x$ of the transmission line in Figure 1.3. The axial current $i(x,t)$ flows through the internal and external resistances $r_i \Delta x$ and $r_o \Delta x$ in series, so Ohm's law gives the voltage drop along the segment, and dividing by Δx and letting $\Delta x \to 0$ gives the axial voltage gradient:

$$v(x+\Delta x,t) - v(x,t) = -\,i(x,t)(r_o + r_i)\Delta x \;\Rightarrow\; \frac{\partial v}{\partial x} = -(r_o + r_i)i \qquad (2.5)$$

At the node, the axial current that leaves the segment is smaller than the current that enters it by the membrane current $i_m \Delta x$ that crosses the membrane, which consists of a capacitive and an ionic part (Kirchhoff's current law):

$$i(x+\Delta x,t) - i(x,t) = -\,i_m \Delta x \;\Rightarrow\; \frac{\partial i}{\partial x} = -\left(c_m \frac{\partial v}{\partial t} + i_{ion}\right) \qquad (2.6)$$

Differentiating Equation (2.5) with respect to x (with r_o and r_i uniform along the axon) and substituting Equation (2.6) eliminates the axial current:

$$\frac{\partial^2 v}{\partial x^2} = -(r_o + r_i)\frac{\partial i}{\partial x} = (r_o + r_i)\left(c_m \frac{\partial v}{\partial t} + i_{ion}\right) \qquad (2.7)$$

Solving for the time derivative, adding the stimulus and using $i_{ion}/c_m = J_{ion}/C_m$ (both per-length quantities contain the factor $2\pi a$) gives the explicit form that is integrated by the program:

$$\frac{\partial v}{\partial t} = \frac{1}{(r_o + r_i)c_m}\frac{\partial^2 v}{\partial x^2} + \frac{J_{in} - J_{ion}}{C_m} = D\frac{\partial^2 v}{\partial x^2} + \frac{J_{in} - J_{ion}}{C_m} \qquad (2.8)$$

with $J_{ion} = J_{Na} + J_K + J_L$ for the active cable and $J_{ion} = g_m v$ for the passive (lecturer's RC) cable.

Step 2, central difference for the spatial derivative. On the grid $x_j = jh$, $t^n = nk$, write the Taylor series of v about x_j at a fixed time level n for a step $+h$ and a step $-h$:

$$v_{j+1}^n = v_j^n + h\,v_x + \frac{h^2}{2}v_{xx} + \frac{h^3}{6}v_{xxx} + \frac{h^4}{24}v_{xxxx} + O(h^5) \qquad (2.9)$$

$$v_{j-1}^n = v_j^n - h\,v_x + \frac{h^2}{2}v_{xx} - \frac{h^3}{6}v_{xxx} + \frac{h^4}{24}v_{xxxx} + O(h^5) \qquad (2.10)$$

Adding the two series cancels all odd derivatives:

$$v_{j+1}^n + v_{j-1}^n = 2v_j^n + h^2 v_{xx} + \frac{h^4}{12}v_{xxxx} + O(h^6) \qquad (2.11)$$

and solving for v_{xx} gives the central difference with a truncation error of second order in h:

$$\left.\frac{\partial^2 v}{\partial x^2}\right|_j^n = \frac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{h^2} - \frac{h^2}{12}v_{xxxx} \approx \frac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{h^2} \qquad (2.12)$$

Step 3, forward difference for the time derivative. The Taylor series in time at node j is

$$v_j^{n+1} = v_j^n + k\, v_t + \frac{k^2}{2} v_{tt} + O(k^3) \qquad (2.13)$$

so the forward difference is accurate to first order in k:

$$\left.\frac{\partial v}{\partial t}\right|_j^n = \frac{v_j^{n+1} - v_j^n}{k} - \frac{k}{2} v_{tt} \approx \frac{v_j^{n+1} - v_j^n}{k} \qquad (2.14)$$

Step 4, substitution and the explicit update. Substituting Equation (2.12) and (2.14) into Equation (2.8) at the point (x_j, t^n) and evaluating the membrane current at the known level n gives

$$\frac{v_j^{n+1} - v_j^n}{k} = D\, \frac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{h^2} + \frac{J_{in,j}^n - J_{ion,j}^n}{C_m} \qquad (2.15)$$

and, multiplying by k, the explicit update used by the program:

$$v_j^{n+1} = v_j^n + r\left(v_{j+1}^n - 2v_j^n + v_{j-1}^n\right) + \frac{k}{C_m}\left(J_{in,j}^n - J_{ion,j}^n\right), \qquad r = \frac{Dk}{h^2} = \frac{k}{(r_o + r_i)c_m h^2} \qquad (2.16)$$

For the passive cable of the lecture, $J_{ion,j}^n = g_m v_j^n$ and Equation (2.16) becomes

$$v_j^{n+1} = v_j^n + \frac{k}{h^2(r_o + r_i)c_m}\left(v_{j+1}^n - 2v_j^n + v_{j-1}^n\right) - \frac{k\, g_m}{C_m} v_j^n \qquad (2.17)$$

in which the leakage term uses the voltage of the same node j. (The transcribed lecturer code uses `vm[0,i]`, the voltage of the first node, in this term this is most likely a transcription error and was not reproduced.) The local truncation error of the scheme is the sum of the two neglected terms, $\tau_j^n = -k/2\, v_{tt} - D\, h^2/12\, v_{xxxx} = O(k + h^2)$.

Step 5, gates of the active cable. In the active cable every node carries its own gates, which are advanced with the same forward difference applied to Equation (1.5), using the voltage of level n:

$$x_j^{n+1} = x_j^n + k\left[\alpha_x\left(v_j^n\right)\left(1 - x_j^n\right) - \beta_x\left(v_j^n\right)x_j^n\right], \qquad x \in \{m, h, n\} \qquad (2.18)$$

Step 6, boundary conditions. The ends of the axon are sealed, which means that no axial current leaves the axon there. By Equation (2.5) this is $\partial v / \partial x = 0$. With a central difference across the boundary this condition introduces ghost nodes $v_{-1}^n = v_1^n$ and $v_N^n = v_{N-2}^n$, and substituting them into Equation (2.16) gives the end-node updates

$$v_0^{n+1} = v_0^n + 2r(v_1^n - v_0^n) + \frac{k}{C_m}(J_{in} - J_{ion})_0^n, \qquad v_{N-1}^{n+1} = v_{N-1}^n + 2r(v_{N-2}^n - v_{N-1}^n) + \frac{k}{C_m}(J_{in} - J_{ion})_{N-1}^n \qquad (2.19)$$

Step 7, stability. Inserting a single Fourier mode $v_j^n = \xi^n e^{\mathrm{i}\beta jh}$ into the diffusion part of Equation (2.16) gives the amplification factor

$$\xi = 1 + r\left(e^{\mathrm{i}\beta h} - 2 + e^{-\mathrm{i}\beta h}\right) = 1 + r(2\cos\beta h - 2) = 1 - 4r\sin^2\frac{\beta h}{2} \qquad (2.20)$$

Since $0 \le \sin^2(\beta h/2) \le 1$, ξ lies between $1 - 4r$ and 1, and the requirement $|\xi| \le 1$ for every wave number reduces to $1 - 4r \ge -1$:

$$r = \frac{Dk}{h^2} \le \frac{1}{2} \qquad \Leftrightarrow \qquad k \le k_{max} = \frac{h^2}{2D} \qquad (2.21)$$

For the squid axon of Problem 3 ($a = 238$ µm, $R_i = 35.4$ Ω·cm, $r_o = 0$), $r_i = R_i/(\pi a^2) = 19{,}893$ Ω/cm and $c_m = 2\pi a C_m = 0.1495$ µF/cm. Because 1 Ω·µF = 1 µs, $D = 1/(r_i c_m) = 3.362 \times 10^{-4}$ cm²/µs $= 0.3362$ cm²/ms, and with $h = 1$ mm and $k = 0.01$ ms the diffusion number is $r = 0.336 < 0.5$, the largest stable step at this h is $k_{max} = 0.0149$ ms. These numbers are reproduced exactly in the result memo of the program (Figure 2.7). The derivation of this section is also available in the program as sheet 5 ("Central/forward difference (Problem 2)"), opened from tab 5 or tab 6.

2.2.4 Problem 3. Program for Action Potential Propagation

`DefaultCableParams` sets the squid giant axon of Hodgkin and Huxley [1] ($a = 238$ µm, $R_i = 35.4$ Ω·cm, $r_o \approx 0$ for a large external bath), a 6 cm axon with $h = 1$ mm, $k = 0.01$ ms, a 12 ms run and a stimulus of -150 µA/cm² for 0.5 ms applied to the first 2 mm. `SimulateCable` is the implementation of Section 2.2.3. It validates the geometry, computes r_i, c_m, D (with the factor 1000 that converts cm²/µs into cm²/ms), the diffusion number r and the passive length constant, and stops before integrating if Equation (2.21) is violated. It then sets every node to the resting state and marches in time: for every node it forms the central difference of Equation (2.12), using the sealed-end form of Equation (2.19) at $j = 0$ and $j = N - 1$, it evaluates J_{ion} either from the HH currents with the node's own gates, which are advanced by Equation (2.18), or as $g_m v$ in the passive mode, and it writes the new voltage of Equation (2.16) into a second array, so that the whole new level is computed from level n only, as the explicit scheme requires. After each step it records the peak V_m and the first upward 0 mV crossing of every node, interpolated linearly between the two-time levels that bracket it, and stores a frame for plotting. Finally, the conduction velocity is computed from the crossing times at 25 % and 75 % of the axon length:

$$\theta = \frac{x_{75\%} - x_{25\%}}{t_c(x_{75\%}) - t_c(x_{25\%})} \cdot 10 \text{ m/s} \qquad (1 \text{ cm/ms} = 10 \text{ m/s}) \qquad (2.22)$$

Measuring between the 25 % and 75 % points keeps both the stimulated region and the sealed end, where the action potential is distorted, out of the velocity estimate.

```
function DefaultCableParams: TCableParams;
begin
  Result.RadiusUm := 238.0;      // squid giant axon (Hodgkin & Huxley, 19
52)
  Result.RiOhmCm := 35.4;        // axoplasm resistivity (ohm.cm)
  Result.RoOhmPerCm := 0.0;      // large external bath, ro << ri
  Result.LengthCm := 6.0;
  Result.DxCm := 0.1;
  Result.Dt := 0.01;
  Result.TEnd := 12.0;
  Result.StimAmp := -150.0;
  Result.StimDur := 0.5;
  Result.StimLengthCm := 0.2;
  Result.Mode := cmActiveHH;
  Result.GmPassive := 0.677;
  Result.SaveEvery := 2;
end;

procedure SimulateCable(const HP: THHParams; const CP: TCableParams;
  out R: TCableResult);
var
  A, Phi, T, Lap, Jin, JNa, JK, JL, Jion, VmNow, InvH2: Double;
  V, VNew, Mg, Hg, Ng, VmPrev: TDoubleArray;
  Stimulated: array of Boolean;
  J, K, NSteps, NFrames, Frame, I1, I2: Integer;
  S: THHState;
  Rt: THHRates;

  procedure SaveFrame(const Idx: Integer; const Time: Double);
  var
    Jj: Integer;
  begin
    R.T[Idx] := Time;
    SetLength(R.Vm[Idx], R.NNodes);
    for Jj := 0 to R.NNodes - 1 do
      R.Vm[Idx][Jj] := AbsoluteVm(V[Jj]);
  end;

begin
  if (CP.DxCm <= 0.0) or (CP.LengthCm <= CP.DxCm) then
    raise EArgumentException.Create('Require 0 < dx < axon length.');
  if CP.RadiusUm <= 0.0 then
    raise EArgumentException.Create('Axon radius must be positive.');
  if CP.RiOhmCm + CP.RoOhmPerCm <= 0.0 then
    raise EArgumentException.Create('ri + ro must be positive.');
  CheckStepArguments(CP.Dt, CP.TEnd);

  // ---- cable constants ----
  A := CP.RadiusUm * 1.0E-4;                          // cm
  R.RiPerLen := CP.RiOhmCm / (Pi * A * A);            // ohm/cm
  R.CmPerLen := HP.Cm * 2.0 * Pi * A;                 // uF/cm
  // ohm.uF = us, so 1/((ri+ro) cm) is in cm2/us; x1000 gives cm2/ms
  R.D := 1000.0 / ((R.RiPerLen + CP.RoOhmPerCm) * R.CmPerLen);
  R.R := R.D * CP.Dt / Sqr(CP.DxCm);
```

```
  if CP.GmPassive > 0.0 then
    // lambda^2 = rm/(ri+ro), rm = 1/(gm 2 pi a), gm in S/cm2
    R.LambdaCm := Sqrt(1.0 / (CP.GmPassive * 1.0E-3 * 2.0 * Pi * A *
      (R.RiPerLen + CP.RoOhmPerCm)))
  else
    R.LambdaCm := NaN;
  R.VelocityMs := NaN;
  R.NNodes := Round(CP.LengthCm / CP.DxCm) + 1;
  R.Stable := R.R <= 0.5;
  R.NSaved := 0;
  SetLength(R.X, R.NNodes);
  for J := 0 to R.NNodes - 1 do
    R.X[J] := J * CP.DxCm;
  if not R.Stable then
    Exit;  // caller reports the violated condition k <= h^2/(2D)

  // initial state: every node at rest
  Phi := TemperatureFactor(HP);
  S := SteadyState(0.0, Phi);
  SetLength(V, R.NNodes);
  SetLength(VNew, R.NNodes);
  SetLength(Mg, R.NNodes);
  SetLength(Hg, R.NNodes);
  SetLength(Ng, R.NNodes);
  SetLength(VmPrev, R.NNodes);
  SetLength(Stimulated, R.NNodes);
  SetLength(R.PeakVm, R.NNodes);
  SetLength(R.TCross, R.NNodes);
  for J := 0 to R.NNodes - 1 do
  begin
    V[J] := 0.0;
    Mg[J] := S.M;
    Hg[J] := S.H;
    Ng[J] := S.N;
    VmPrev[J] := AbsoluteVm(0.0);
    R.PeakVm[J] := VmPrev[J];
    R.TCross[J] := NaN;
    Stimulated[J] := R.X[J] <= CP.StimLengthCm + 1.0E-9;
  end;

  NSteps := Round(CP.TEnd / CP.Dt);
  if CP.SaveEvery < 1 then
    NFrames := NSteps + 1
  else
    NFrames := NSteps div CP.SaveEvery + 1;
  SetLength(R.T, NFrames);
  SetLength(R.Vm, NFrames);
  SaveFrame(0, 0.0);
  Frame := 1;
  InvH2 := 1.0 / Sqr(CP.DxCm);

  // time marching (forward difference)
  for K := 0 to NSteps - 1 do
  begin
    T := K * CP.Dt;
    for J := 0 to R.NNodes - 1 do
    begin
```

```
      // central difference with sealed-end ghost nodes
      if J = 0 then
        Lap := (2.0 * V[1] - 2.0 * V[0]) * InvH2
      else if J = R.NNodes - 1 then
        Lap := (2.0 * V[J - 1] - 2.0 * V[J]) * InvH2
      else
        Lap := (V[J + 1] - 2.0 * V[J] + V[J - 1]) * InvH2;

      if Stimulated[J] and (T < CP.StimDur) then
        Jin := CP.StimAmp
      else
        Jin := 0.0;

      if CP.Mode = cmActiveHH then
      begin
        S.V := V[J];
        S.M := Mg[J];
        S.H := Hg[J];
        S.N := Ng[J];
        IonicCurrents(HP, S, JNa, JK, JL);
        Jion := JNa + JK + JL;
        Rt := ComputeRates(V[J], Phi);
        Mg[J] := Mg[J] + CP.Dt * (Rt.Am * (1.0 - Mg[J]) - Rt.Bm * Mg[J]);
        Hg[J] := Hg[J] + CP.Dt * (Rt.Ah * (1.0 - Hg[J]) - Rt.Bh * Hg[J]);
        Ng[J] := Ng[J] + CP.Dt * (Rt.An * (1.0 - Ng[J]) - Rt.Bn * Ng[J]);
      end
      else
        Jion := CP.GmPassive * V[J];

      VNew[J] := V[J] + CP.Dt * (R.D * Lap + (Jin - Jion) / HP.Cm);
    end;

    for J := 0 to R.NNodes - 1 do
    begin
      V[J] := VNew[J];
      VmNow := AbsoluteVm(V[J]);
      if VmNow > R.PeakVm[J] then
        R.PeakVm[J] := VmNow;
      if IsNan(R.TCross[J]) and (VmPrev[J] < SPIKE_LEVEL) and
         (VmNow >= SPIKE_LEVEL) then
        // linear interpolation of the crossing instant
        R.TCross[J] := T + CP.Dt * (SPIKE_LEVEL - VmPrev[J]) /
          (VmNow - VmPrev[J]);
      VmPrev[J] := VmNow;
    end;

    if (CP.SaveEvery >= 1) and (((K + 1) mod CP.SaveEvery) = 0) and
       (Frame < NFrames) then
    begin
      SaveFrame(Frame, (K + 1) * CP.Dt);
      Inc(Frame);
    end
    else if (CP.SaveEvery < 1) and (Frame < NFrames) then
    begin
      SaveFrame(Frame, (K + 1) * CP.Dt);
      Inc(Frame);
    end;
```

```
  end;
  R.NSaved := Frame;

  // conduction velocity between 25 % and 75 % of the axon length
  I1 := Round(0.25 * (R.NNodes - 1));
  I2 := Round(0.75 * (R.NNodes - 1));
  if (I2 > I1) and (not IsNan(R.TCross[I1])) and (not IsNan(R.TCross[I2]))
and
     (R.TCross[I2] > R.TCross[I1]) then
    // 1 cm/ms = 10 m/s
    R.VelocityMs := 10.0 * (R.X[I2] - R.X[I1]) / (R.TCross[I2] - R.TCross[
I1]);
end;
```

The model exhibits propagation because of the coupling term $D\ \partial^2 v/\partial x^2$ together with the regenerative membrane current. When node j fires, its depolarization makes the central difference at node $j+1$ negative (in the lecturer's sign), so Equation (2.16) drives v_{j+1} toward depolarization. As soon as the neighbour reaches threshold, its own sodium current takes over and produces a full action potential, which in turn depolarizes node $j+2$. The wave therefore travels with constant amplitude, set by the membrane, and at a constant velocity, set by how fast the axial current can charge the next segment, that is by D and the membrane kinetics. In the passive mode the same coupling only spreads and attenuates the stimulated depolarization with the length constant λ, and no action potential appears away from the stimulation site. Comparing both modes in the same program is the direct demonstration asked for in Problem 3.

On the form, `FillAPPEditor` fills the parameter table with lengths in millimetres, and the two preset buttons select either the squid axon (`btnPresetSquidClick`) or a lecturer-scale fibre (`btnPresetLecturerClick`). The lecturer's and the senior's programs show a 2 mm axon. A squid axon conducting at about 12 m/s with an action potential lasting about 2 ms produces a wave about 2.4 cm long, so a 2 mm squid axon would be shorter than the action potential itself. The lecturer-scale preset therefore uses the same equations with a thin fibre ($a = 1$ µm, 2 mm, $h = 0.1$ mm, recording sites every 0.4 mm), whereas the squid preset is kept as the default because its velocity can be compared with the 1952 measurements. `btnRunAPPClick` reads the parameters, converts millimetres to centimetres, calls `SimulateCable`, reports a violated stability condition together with k_{max}, and then calls `PlotCable` (Vm(t) at evenly spaced recording sites and Vm(x) at evenly spaced instants), `ReportCable` (cable constants, r, λ, θ and a table of peak voltages and crossing times) and `UpdateAPPFrame`, which moves the time cursor, the bold Vm(x) profile and the overlay of the space-time view when the time slider or the Play button changes the frame. The space-time view offers a heat map, a 3D waterfall and a 3D mesh of $V_m(x,t)$. `OpenSheet` and `ShowEquationSheet` open the derivation windows described in Section 2.2.1.

```
procedure TForm1.FillAPPEditor(const ACable: TCableParams; ASites: Integer
);
begin
```

```
  // lengths are shown in mm; RoundTo removes binary noise of cm * 10
  SetupEditor(vleAPP);
  AddParam(vleAPP, K_CA, ACable.RadiusUm);
  AddParam(vleAPP, K_CRI, ACable.RiOhmCm);
  AddParam(vleAPP, K_CRO, ACable.RoOhmPerCm);
  AddParam(vleAPP, K_CLEN, RoundTo(ACable.LengthCm * C_MM_PER_CM, -6));
  AddParam(vleAPP, K_CDX, RoundTo(ACable.DxCm * C_MM_PER_CM, -6));
  AddParam(vleAPP, K_CDT, ACable.Dt);
  AddParam(vleAPP, K_CTEND, ACable.TEnd);
  AddParam(vleAPP, K_CJ, ACable.StimAmp);
  AddParam(vleAPP, K_CDUR, ACable.StimDur);
  AddParam(vleAPP, K_CSL, RoundTo(ACable.StimLengthCm * C_MM_PER_CM, -6));
  // gm of the passive cable = chord conductance of the resting HH membran
e
  AddParam(vleAPP, K_CGM, RoundTo(RestingConductance(DefaultHHParams), -4)
);
  AddParam(vleAPP, K_CSITES, ASites);
end;

procedure TForm1.btnPresetSquidClick(Sender: TObject);
begin
  StopPlay;
  FillAPPEditor(DefaultCableParams, C_SITES_DEFAULT);
  SetStatus('Preset: squid giant axon (a = 238 um, 60 mm axon, h = 1 mm).
' +
    'Press Run APP simulation.');
end;

procedure TForm1.btnPresetLecturerClick(Sender: TObject);
var
  LCable: TCableParams;
begin
  // Lecturer/senior scale: 2 mm axon with recording sites every 0.4 mm.
  // A thin fibre (a = 1 um) gives an AP that fits into 2 mm; the cable
  // equation and the HH membrane are unchanged (see ISSUES, ISS-023).
  StopPlay;
  LCable := DefaultCableParams;
  LCable.RadiusUm := 1.0;
  LCable.LengthCm := 0.2;        // 2 mm
  LCable.DxCm := 0.01;           // h = 0.1 mm -> r = 0.14 (stable)
  LCable.Dt := 0.01;
  LCable.TEnd := 6.0;
  LCable.StimLengthCm := 0.02;   // 0.2 mm
  FillAPPEditor(LCable, C_SITES_DEFAULT);
  SetStatus('Preset: lecturer scale (a = 1 um, 2 mm axon, sites every ' +
    '0.4 mm). Press Run APP simulation.');
end;

procedure TForm1.btnRunAPPClick(Sender: TObject);
var
  LParams: THHParams;
  LCable: TCableParams;
  LSites: Integer;
begin
  StopPlay;
  try
    LParams := ReadModelParams;
```

```
    LCable := DefaultCableParams;
    LCable.RadiusUm := ParamValue(vleAPP, K_CA);
    LCable.RiOhmCm := ParamValue(vleAPP, K_CRI);
    LCable.RoOhmPerCm := ParamValue(vleAPP, K_CRO);
    LCable.LengthCm := ParamValue(vleAPP, K_CLEN) / C_MM_PER_CM;
    LCable.DxCm := ParamValue(vleAPP, K_CDX) / C_MM_PER_CM;
    LCable.Dt := ParamValue(vleAPP, K_CDT);
    LCable.TEnd := ParamValue(vleAPP, K_CTEND);
    LCable.StimAmp := ParamValue(vleAPP, K_CJ);
    LCable.StimDur := ParamValue(vleAPP, K_CDUR);
    LCable.StimLengthCm := ParamValue(vleAPP, K_CSL) / C_MM_PER_CM;
    LCable.GmPassive := ParamValue(vleAPP, K_CGM);
    LSites := EnsureRange(ParamInt(vleAPP, K_CSITES), 2, 8);
    if rgCableMode.ItemIndex = 1 then
      LCable.Mode := cmPassive
    else
      LCable.Mode := cmActiveHH;
    // keep roughly 600 stored frames for plotting and animation
    LCable.SaveEvery := Max(1, Round(LCable.TEnd / LCable.Dt) div 600);

    Screen.Cursor := crHourGlass;
    try
      SimulateCable(LParams, LCable, FCable);
    finally
      Screen.Cursor := crDefault;
    end;
    Inc(FCableRunId);   // invalidates the cached 3D surface

    if not FCable.Stable then
    begin
      FCableValid := False;
      RefreshSpaceTime;
      ShowError(Format('Explicit scheme unstable: r = k/((ro+ri) cm h^2) =
%s ' +
        '> 0.5. Use k <= %s ms or a larger h (mm).', [FloatText(FCable.R,
3),
        FloatText(0.5 * Sqr(LCable.DxCm) / FCable.D, 5)]));
      Exit;
    end;
    FCableValid := True;
    FAPPFrame := 0;
    PlotCable(LSites);
    BuildHeatMap;
    ReportCable(LCable);
    tbAPPTime.Max := Max(1, FCable.NSaved - 1);
    tbAPPTime.Position := 0;
    UpdateAPPFrame;
    SetStatus(Format('APP finished: %d nodes, %d frames. Press Play or mov
e ' +
      'the time slider.', [FCable.NNodes, FCable.NSaved]));
  except
    on E: Exception do
      ShowError(E.Message);
  end;
end;
```

```
procedure TForm1.PlotCable(ASites: Integer);
var
  LSeries: TFastLineSeries;
  LIdx, LK, LNode, LFrame: Integer;
begin
  ClearChart(chtAPPTime);
  ClearChart(chtAPPSpace);
  // Vm(t) at evenly spaced recording sites, coloured by position
  for LIdx := 0 to ASites - 1 do
  begin
    LNode := Round(LIdx * (FCable.NNodes - 1) / (ASites - 1));
    LSeries := AddLine(chtAPPTime, Format('x = %s mm',
      [FloatText(FCable.X[LNode] * C_MM_PER_CM, 2)]),
      OrderedLineColor(LIdx, ASites));
    for LK := 0 to FCable.NSaved - 1 do
      LSeries.AddXY(FCable.T[LK], FCable.Vm[LK][LNode]);
  end;
  // time cursor (vertical line), excluded from the legend
  FTimeMarker := TLineSeries.Create(chtAPPTime);
  FTimeMarker.ParentChart := chtAPPTime;
  FTimeMarker.Title := 'time cursor';
  FTimeMarker.SeriesColor := C_STIM;
  FTimeMarker.ShowInLegend := False;
  FTimeMarker.Pointer.Visible := False;
  FinishChart(chtAPPTime);

  // Vm(x) at evenly spaced instants (thin, coloured by time) + live profi
le
  for LIdx := 1 to ASites do
  begin
    LFrame := Round(LIdx * (FCable.NSaved - 1) / (ASites + 1));
    LSeries := AddLine(chtAPPSpace, Format('t = %s ms',
      [FloatText(FCable.T[LFrame], 2)]), OrderedLineColor(LIdx - 1, ASites
));
    for LK := 0 to FCable.NNodes - 1 do
      LSeries.AddXY(FCable.X[LK] * C_MM_PER_CM, FCable.Vm[LFrame][LK]);
  end;
  FLiveProfile := AddLine(chtAPPSpace, 'time slider', C_BLACK);
  FLiveProfile.Tag := C_TAG_EMPH;
  FinishChart(chtAPPSpace);
end;

procedure TForm1.ReportCable(const AParams: TCableParams);
var
  LLines: TStrings;
  LIdx, LNode: Integer;
begin
  LLines := memAPP.Lines;
  LLines.BeginUpdate;
  try
    LLines.Clear;
    if AParams.Mode = cmActiveHH then
      LLines.Add('Mode: ACTIVE cable (HH membrane at every node)')
    else
      LLines.Add(Format('Mode: PASSIVE RC cable (gm = %s mS/cm2)',
        [FloatText(AParams.GmPassive, 4)]));
    LLines.Add(Format('ri = %s ohm/cm, cm = %s uF/cm',
```

```
      [FloatText(FCable.RiPerLen, 1), FloatText(FCable.CmPerLen, 5)]));
    LLines.Add(Format('D = 1/((ro+ri) cm) = %s cm2/ms', [FloatText(FCable.
D, 5)]));
    LLines.Add(Format('r = D k / h^2 = %s (stable, limit 0.5)',
      [FloatText(FCable.R, 4)]));
    LLines.Add(Format('Passive length constant lambda = %s mm',
      [FloatText(FCable.LambdaCm * C_MM_PER_CM, 4)]));
    LLines.Add(Format('Conduction velocity (25%%-75%% of length) = %s m/s'
,
      [FloatText(FCable.VelocityMs, 3)]));
    LLines.Add('');
    LLines.Add('   x(mm)   peak Vm(mV)   t(Vm=0 mV)(ms)');
    for LIdx := 0 to 10 do
    begin
      LNode := Round(LIdx * (FCable.NNodes - 1) / 10);
      LLines.Add(Format('%7s   %10s   %12s',
        [FloatText(FCable.X[LNode] * C_MM_PER_CM, 2),
        FloatText(FCable.PeakVm[LNode], 2), FloatText(FCable.TCross[LNode]
, 3)]));
    end;
  finally
    LLines.EndUpdate;
  end;
end;

procedure TForm1.UpdateAPPFrame;
var
  LIdx: Integer;
  LTime: Double;
begin
  if not FCableValid then
    Exit;
  FAPPFrame := EnsureRange(tbAPPTime.Position, 0, FCable.NSaved - 1);
  LTime := FCable.T[FAPPFrame];
  lblAPPTime.Caption := Format('t = %s ms', [FloatText(LTime, 2)]);
  if FLiveProfile <> nil then
  begin
    FLiveProfile.BeginUpdate;
    try
      FLiveProfile.Clear;
      for LIdx := 0 to FCable.NNodes - 1 do
        FLiveProfile.AddXY(FCable.X[LIdx] * C_MM_PER_CM,
          FCable.Vm[FAPPFrame][LIdx]);
      FLiveProfile.Title := Format('time slider: t = %s ms',
        [FloatText(LTime, 2)]);
    finally
      FLiveProfile.EndUpdate;
    end;
  end;
  if FTimeMarker <> nil then
  begin
    FTimeMarker.Clear;
    FTimeMarker.AddXY(LTime, C_VM_LO - 10.0);
    FTimeMarker.AddXY(LTime, C_VM_HI + 10.0);
  end;
```

```
  RefreshSpaceTime;
end;

procedure TForm1.OpenSheet(AIndex: Integer);
begin
  if (AIndex < SHEET_FIRST) or (AIndex > SHEET_LAST) then
    Exit;
  try
    ShowEquationSheet(Self, AIndex);
    SetStatus(Format('Sheet %d opened: %s', [AIndex, SheetTitle(AIndex)]))
;
  except
    on E: Exception do
    begin
      AppendLog('ERROR', 'Sheet ' + IntToStr(AIndex) + ': ' + E.Message);
      ShowError('The sheet could not be opened: ' + E.Message);
    end;
  end;
end;
```

```
procedure    ShowEquationSheet(AOwnerForm:    TForm;    AIndex:    Integer);
var
  LForm:                                                          TSheetForm;
begin
  if    (AIndex    <    SHEET_FIRST)    or    (AIndex    >    SHEET_LAST)    then
    raise EArgumentOutOfRangeException.CreateFmt('Sheet %d does not exist.',
      [AIndex]);
  if            GSheets[AIndex]            <>            nil            then
  begin
    // already open: restore it if minimised and bring it to the front
    if       GSheets[AIndex].WindowState       =       wsMinimized       then
      GSheets[AIndex].WindowState                :=                wsNormal;
    GSheets[AIndex].Show;
    GSheets[AIndex].BringToFront;
    Exit;
  end;
  LForm   :=   TSheetForm.CreateSheet(AOwnerForm,   AIndex,   OpenSheetCount);
  GSheets[AIndex]                              :=                          LForm;
  LForm.Show;
end;
```

2.2.5 Problem 4. The Hodgkin-Huxley Model in Two Research Works

Paper 1: a model of the pacemaking neuron of the respiratory central pattern generator

Amini, Bidani, Zwischenberger and Clark [7] address a central question of respiratory neurophysiology: how much of the breathing rhythm is generated by the intrinsic membrane properties of individual neurons and how much by network connectivity. The respiratory central pattern generator is a population of medullary neurons that drives the phrenic motor neurons and hence the diaphragm. Its putative pacemaker, the pre-inspiratory neuron of the

pre-Bötzinger complex (PBC), fires before inspiration and sets the respiratory rate. The authors build an HH-type model of this neuron as the driving element of a larger network model and use it to dissect the ionic mechanism of bursting and the phase sensitivity of the rhythm.

The first important aspect is that the paper keeps the complete Hodgkin-Huxley formalism and only enlarges the set of conductances. Every voltage-gated current is written as a maximal conductance multiplied by powers of gating variables and a driving force, for example $I_{CaT} = g_{CaT} d_T^2 f_T (v - 37.0)$, and every gating variable follows first-order kinetics with $x_\infty = \alpha/(\alpha + \beta)$ and $\tau_x = c/(\alpha + \beta)$ [7, Table II]. Several rate functions have exactly the form $a(v + b)/\left(e^{c(v+b)} - 1\right)$ of α_m and α_n in the original model [1] and therefore inherit the same removable singularity that is handled analytically by `ExpRatio` in this assignment. Instead of the three currents of the squid axon, the PBC neuron contains fast and persistent sodium currents (I_{Na}, I_{NaP}), T-, N- and L-type calcium currents, delayed-rectifier and transient potassium currents (I_K, I_A, I_D), a calcium-activated potassium current (I_{KCa}), a hyperpolarization-activated cation current (I_h), pump and exchanger currents and an intracellular calcium balance [7, Fig. 2]. The second aspect is an explicit temperature correction: because voltage-clamp data of the different currents were recorded at different temperatures, time constants and maximal conductances were scaled by $Q_{10}^{(T_1 - T_2)/10}$ to 302 K with current-specific Q_{10} values between 1.28 and 5.1 [7, Table I]. This is the same mechanism as the factor $\phi = 3^{(T-6.3)/10}$ of Hodgkin and Huxley [1], which the simulator of this assignment exposes as its temperature parameter. The third aspect is numerical: the stiff system was integrated with the variable-step CVODE solver rather than with a fixed-step Runge-Kutta scheme.

The model reproduces the interburst trajectory, the rising baseline during a burst, the burst duration and period and the peak intraburst frequency of recorded PBC neurons [7, Fig. 2(a)]. Its main value lies in in-silico channel-block experiments. Removing I_{NaP} abolishes bursting and leaves tonic spiking at about 15 Hz, so the persistent sodium current sets the depolarized baseline on which bursts ride. I_{Na} generates the spikes and recruits I_{CaL} and I_{CaN}, the resulting rise of intracellular calcium activates I_{KCa}, which terminates the burst [7, Fig. 3]. Phasic current pulses applied at different phases of the cycle give a phase response curve: stimulation during the burst produces a phase delay that decreases and reverses into an advance, while stimulation during the repolarizing phase gives a plateau of constant phase advance [7, Fig. 4]. Two inconsistencies should be noted when citing the paper. The Methods section specifies a 150 nA stimulus, whereas the caption of Fig. 4 states 100 nA, and the phase change is defined as $\Delta P = P_0 - P_1$ with positive values said to indicate a delay, although a delay lengthens the perturbed period ($P_1 > P_0$) and gives a negative ΔP, as the Results text itself states. The authors also note that several potassium kinetics were adapted from neighbouring respiratory neurons because direct voltage-clamp data were scarce.

Paper 2: high-speed, low-resource digital implementation of the HH model using base-2 functions

Haghiri, Naderi, Ghanbari and Ahmadi [8] treat the HH model as the most biologically faithful but also the most expensive neuron model for neuromorphic hardware: the powers m^3 and n^4, divisions and exponential rate functions require multipliers, dividers and look-up resources that limit how many neurons fit on a chip. The paper keeps the four-state structure of the model and replaces every costly nonlinear term with a short linear combination of base-2 exponentials, which digital logic evaluates with shifters, adders and subtractors only. The mathematical core is a projection onto an orthonormal basis obtained by Gram-Schmidt orthonormalization of $\{2^{-2v}, 2^{-v}, 1, 2^{v}, 2^{2v}\}$ on $[0,1]$, each term is approximated by its first five generalized Fourier coefficients after the voltage is normalized to the unit interval [8]. A detail directly relevant to this assignment is the sign convention: Haghiri et al. use the modern form with $V_{Na} = 115$ mV, $V_K = -12$ mV and $\alpha_m = (2.5 - 0.1V)/(e^{2.5-0.1V} - 1)$, which is exactly the lecturer's form with $v = -V$. The two descriptions are therefore the same model, and the alignment $V_m = -(v + 70)$ of Problem 1 equals $V_m = V - 70$ in their notation.

The approximation is evaluated at three levels. Time-domain traces for stimuli of 2 to 80 µA/cm² agree with the original model with a mean correlation of 90.4 %, mean RMSE of 7.4 mV and mean NRMSE of 9.34 % [8, Table II]. Dynamically, the reduced system keeps the type and nearly the location of its fixed points, which the authors show with nullclines, Jacobian matrices and phase portraits. In hardware the model is discretized with the forward Euler method ($\Delta t = 2^{-7}$), pipelined with 30-bit fixed-point words and implemented on a Virtex-4 FPGA using 4 % of the slice flip-flops, 19 % of the LUTs, no hardware multipliers and a maximum clock of 200 MHz [8, Table III]. Some points require a critical reading. The error evaluation is reported for "dt = 10 ms", which cannot integrate HH kinetics and is presumably a lost unit prefix, the same defect as the "10 sec" step of the lecture slide (Section 1.3). The single action potential shown for 2 µA/cm² is most likely an initial-condition artefact, because the reference traces start with the gates near zero, with the simulator of this assignment, zero initial gates produce one spike at 4.1 ms for $J_{in} = -2$ µA/cm² and even at 5.4 ms without any input, whereas a membrane started at rest does not fire at this level. The paper also does not explain how the standard HH equations, which do not burst intrinsically, produce the bursting patterns of its Fig. 9, and its introduction dates the Nobel Prize of Hodgkin and Huxley to 1961 instead of 1963.

Table 2.1. *Comparison of the HH model in this assignment and in the two reviewed papers*

Aspect	**This assignment**	**Amini et al. [7]**	**Haghiri et al. [8]**
Purpose	Teaching APG, refractoriness, propagation	Mechanism of the respiratory rhythm	Low-cost neuromorphic hardware
Currents	I_{Na}, I_K, I_L (squid axon)	More than a dozen, incl. I_{NaP}, I_{Ca}, I_{KCa}, I_h	I_{Na}, I_K, I_L (squid axon)
Sign convention	HH 1952, $V_m = -(v + 70)$	Absolute physiological mV	Modern, $V = -v$
Temperature	$\phi = 3^{(T-6.3)/10}$	Current-specific Q_{10}, 302 K	Not used
Integration	RK4, $\Delta t = 0.01$ ms, FTCS cable	CVODE, variable step	Forward Euler, fixed point
Nonlinear terms	Exact, singularities handled	Exact (HH form)	Base-2 approximations
Validation	Python cross-check, HH 1952 velocity	Recorded PBC activity	RMSE, phase plane, FPGA traces

Impact of the Hodgkin-Huxley work on medical and biological technology

The deepest impact of Hodgkin and Huxley [1] is methodological. They showed that excitability can be explained quantitatively by a small set of voltage-dependent conductances described by gating variables with first-order kinetics, and that this description, measured under voltage clamp, predicts the action potential, the refractory period and the conduction velocity. The two papers show how far this template travels. In Amini et al. [7] it becomes a tool of respiratory physiology: more than a dozen conductances in exactly the HH formalism reproduce the bursting of the respiratory pacemaker, pharmacological block becomes a computational experiment that identifies the currents that start and stop a breath, and the phase response curve indicates when an electrical stimulus can advance or delay the next breath, which is the knowledge needed for drug targeting and for electrical control of breathing. The same conductance-based approach was carried into cardiac electrophysiology by Noble's modification of the HH equations for Purkinje fibre and pacemaker potentials [9], the starting point of present-day models of drug action on cardiac ion channels. Haghiri et al. [8] illustrate the engineering line of impact that because the HH model remains the reference for biological fidelity, neuromorphic engineers try to keep its dynamics while reducing its cost, and a multiplier-free HH neuron running at 200 MHz on a small FPGA makes real-time networks of biophysically realistic neurons feasible. The authors position this for the study of disorders

such as epilepsy and Parkinson's disease and for bio-hybrid systems, citing neuroprostheses that restore communication between neuronal populations [10] and closed-loop neuroprostheses [11], which require artificial neurons that respond to living tissue on its own millisecond time scale. Together, the papers show the HH model both as a scientific instrument and as an engineering specification. The simulator developed in Problems 1 and 3 sits at the origin of both lines: it solves the same equations, shows the same refractoriness and input-dependent firing rate, and reproduces the propagation velocity reported in 1952.

2.3 Execution Analysis

2.3.1 Action Potential Generation and Output Alignment

Figure 2.3 shows tab 1 for the default step protocol, $J_{in} = -10$ µA/cm² from 1 ms to 61 ms, integrated with the coupled RK4 at $\Delta t = 0.01$ ms. Before the step the membrane stays exactly at $V_m = -70$ mV: the exported data of the program give $J_{Na} = 1.220$, $J_K = -4.400$ and $J_L = 3.184$ µA/cm² at rest, whose sum is zero to within 4×10^{-3} µA/cm², with $m_\infty = 0.0529$, $h_\infty = 0.5961$ and $n_\infty = 0.3177$, which confirms that the initial condition is a true equilibrium. After the onset the membrane fires repetitively. Each action potential rises from about -60 mV to a peak of about $+35$ mV, falls below rest to an after-hyperpolarization of about -80 mV, close to the potassium reversal potential of -82 mV, and then depolarizes slowly toward the next threshold. Thanks to the alignment of Equation (2.2) the trace has the shape and the voltage range of a physiological recording, whereas the raw variable v of the model would show downward spikes from 0 to about -105 mV. The current chart shows the inward sodium current as a short, large pulse at the upstroke followed by the slower outward potassium current, and the gate and conductance charts show m rising almost instantly from 0.05 to nearly 1 while h falls and n rises with a delay, exactly the separation of time scales described in Section 1.2.

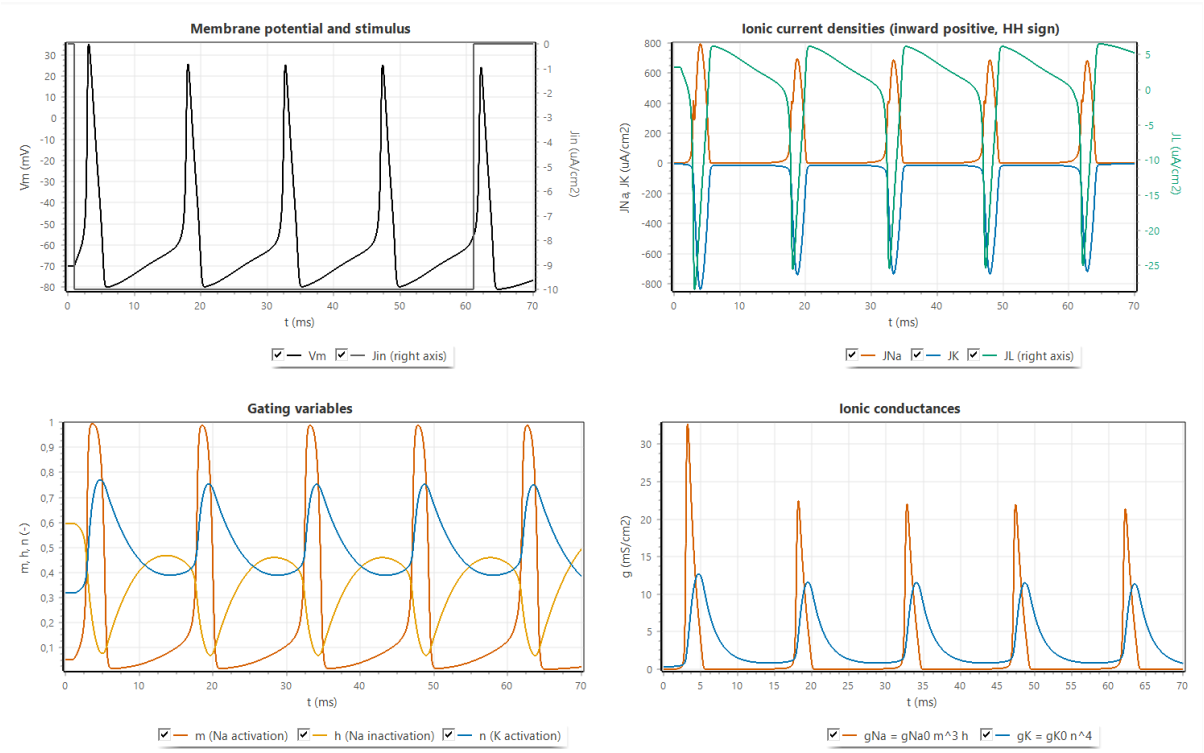


Figure 2.3. *Tab 1 for a step of $J_{in} = -10$ μA/cm²: membrane voltage and stimulus, ionic currents (HH sign, inward positive), gating variables and conductances*

The response to different constant levels is summarized in Table 2.2, obtained from 100 ms runs with the step starting at 1 ms. Levels of -2 μA/cm² or weaker remain subthreshold, -3 to -5 μA/cm² give a single action potential followed by a stable depolarized state, and repetitive firing starts between -6 and -7 μA/cm². Stronger inputs shorten the interval and raise the first peak slightly, until at -100 μA/cm² only one spike occurs and the membrane remains depolarized (depolarization block). The lecturer's staircase protocol produces action potentials at 3.23, 28.36 and 40.01 ms, one at the onset of each new level: the -5 μA/cm² level only elicits a single onset spike (Table 2.2), and the -10 and -20 μA/cm² levels last only 13 and 12 ms, which is not longer than the interval needed for a second spike at these levels. Selecting the lecturer's sequential RK4 changes the spike times by at most 0.01 ms (for example an interval of 17.15 ms instead of 17.14 ms at -7 μA/cm²), so the output of the program agrees with the lecturer's method within the resolution of the plots, and the senior's program, which uses the same equations with a small constant added to the rate denominators, shows qualitatively the same waveforms.

Table 2.2. *Response of the space-clamped membrane to constant input levels (coupled RK4, $\Delta t = 0.01$ ms, step from 1 ms, 100 ms run)*

J_{in} (µA/cm²)	Action potentials	Last ISI (ms)	First peak V_m (mV)
-2	0	-	-
-3	1	-	32.51
-5	1	-	34.05
-7	6	17.14	34.69
-10	7	14.64	35.26
-20	9	11.56	36.30
-40	11	9.21	37.49
-100	1, then block	-	40.00

2.3.2 Gating Phase Plots

Figure 2.4 shows the trajectories of the three gates against V_m for the same run, drawn over the steady-state curves. The trajectory of m stays close to $m_\infty(V_m)$ during the slow phases and separates from it only during the fast upstroke and downstroke, because τ_m is shorter than 0.5 ms, the corresponding loop is narrow. The trajectories of h and n form wide loops that lie far from their steady-state curves, because τ_h and τ_n are several milliseconds: during the spike h and n cannot follow the voltage, and after it they recover only gradually. The first loop starts from the resting values ($h = 0.60$, $n = 0.32$) and is different from the following loops, which start from $h \approx 0.45$ because recovery between spikes is incomplete under the sustained current, this incomplete recovery is the refractoriness that limits the firing rate. Because the dots are plotted at a constant time step, their spacing shows the speed of the state: the dots are dense during the slow interspike depolarization and sparse on the upstroke, where V_m changes by more than 100 mV within about a millisecond.

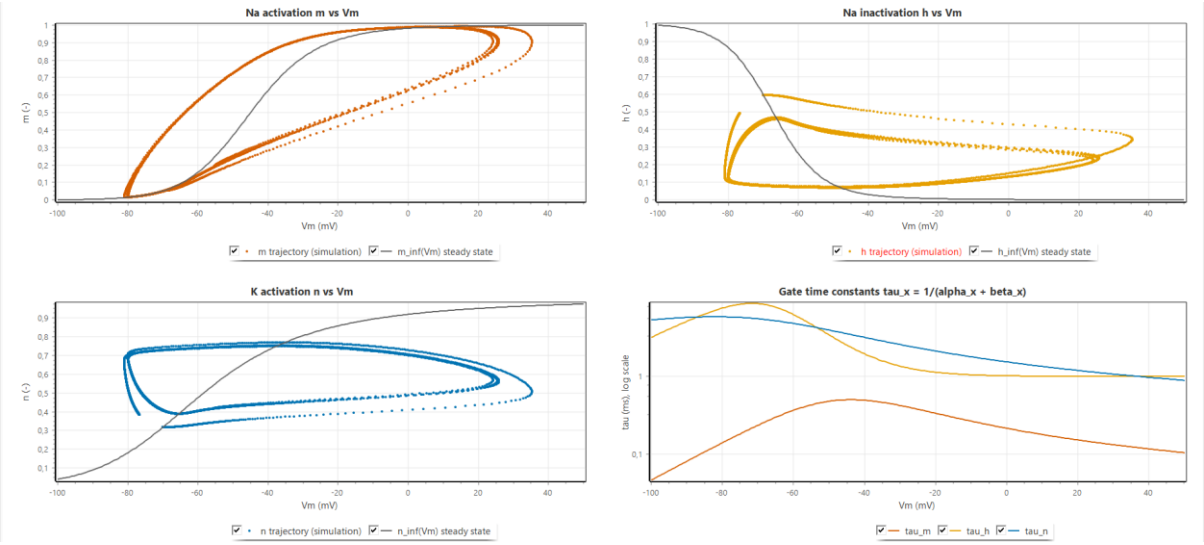


Figure 2.4. *Tab 2: trajectories of* m, h *and* n *against* V_m *(dots at a constant time step) over the steady states* $x_\infty(V_m)$*, and the time constants* $\tau_x(V_m)$

2.3.3 Refractory Period

Figure 2.5 shows the refractory sweep with a conditioning pulse $J_1 = -40$ µA/cm² of 0.5 ms and the test interval varied in steps of 0.5 ms, selected values are listed in Table 2.3. The rested single-pulse threshold is 13.27 µA/cm². For intervals up to 3.5 ms no test pulse up to 500 µA/cm² produces a second action potential, which is the absolute refractory period. At 4.5 ms the threshold is just below the search limit (496 µA/cm²), and it then falls steeply, to 85.8 µA/cm² at 7.5 ms and 37.3 µA/cm² at 10.5 ms, which is the relative refractory period. The threshold returns to within 5 % of the rested value at about 15 ms (13.75 µA/cm² at 15.5 ms). Between about 16.5 and 25 ms the threshold is even lower than at rest, with a minimum of 11.20 µA/cm² at 18.5 ms, about 16 % below the rested value. This supernormal phase is consistent with the gate dynamics, by then n has decayed close to its resting value, while h, which recovered quickly during the after-hyperpolarization, is still slightly above its resting value, so that more sodium conductance is available than at rest. The right axis shows that a second action potential elicited late in the relative refractory period is smaller (30.3 mV at 10.5 ms against 36.3 mV at 17.5 ms), because the available sodium conductance is still reduced. The lower chart shows the corresponding paired-pulse responses for the interval chosen with the track bar.

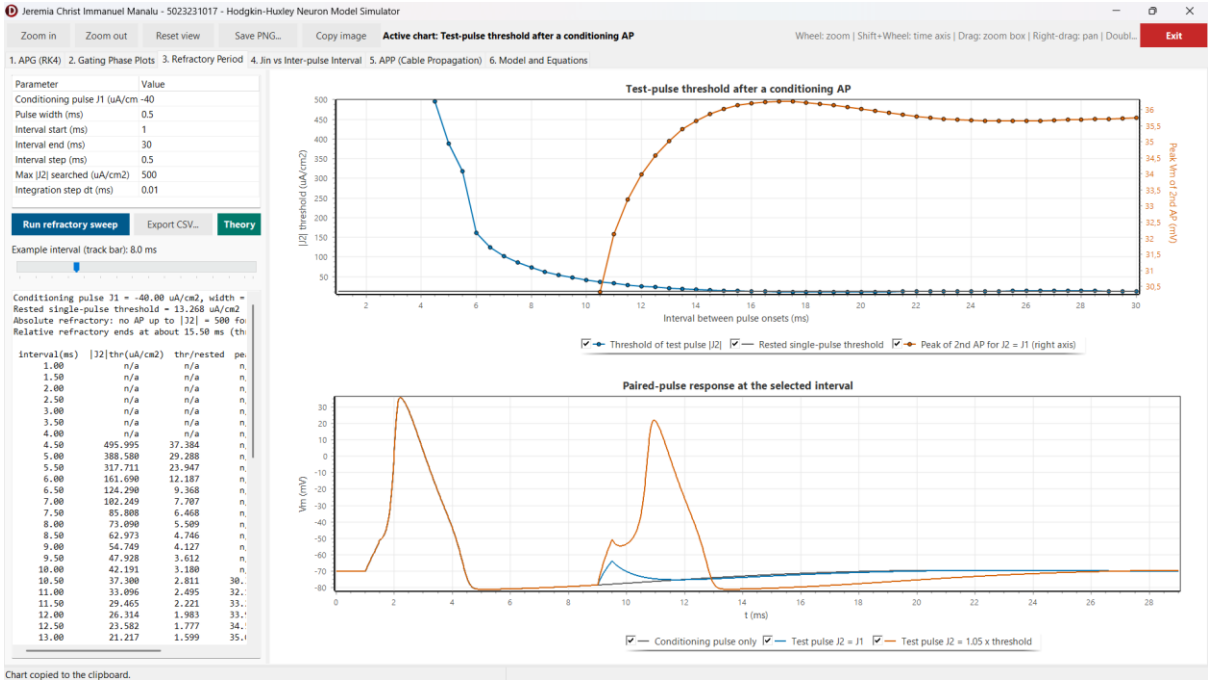


Figure 2.5. *Tab 3: test-pulse threshold and peak of the second action potential against the interval between pulse onsets, and the paired-pulse response at the selected interval*

Table 2.3. *Test-pulse threshold after a conditioning action potential ($J_1 = -40$ μA/cm², width 0.5 ms, rested threshold 13.27 μA/cm²)*

Interval (ms)	Test threshold $\lvert J_2 \rvert_{th}$ (μA/cm²)	Ratio to rested threshold
3.5	none up to 500	-
4.5	496.0	37.4
6.5	124.3	9.37
8.5	63.0	4.75
10.5	37.3	2.81
12.5	23.6	1.78
15.5	13.75	1.04
18.5	11.20	0.84
24.5	13.42	1.01
29.5	13.52	1.02

2.3.4 How the Jin Level Programs the Inter-Pulse Interval

Figure 2.6 and Table 2.4 show the steady firing obtained in tab 4 for constant levels from 0 to −68 μA/cm². Up to −2 μA/cm² the membrane is silent, from −3 to −6 μA/cm² it fires once or twice and then settles, and from −7 μA/cm² it fires tonically with an interval of 17.15 ms (58.3 Hz). The interval then decreases monotonically with $|J_{in}|$, steeply at first and more slowly

at large inputs: 14.64 ms (68.3 Hz) at −10, 11.57 ms (86.5 Hz) at −20, 9.21 ms (108.6 Hz) at −40 and 8.33 ms (120.1 Hz) at −54 μA/cm². At −55 μA/cm² and beyond, firing stops after two or three spikes because the membrane stays depolarized and sodium inactivation does not recover (depolarization block). The input level therefore programs the inter-pulse interval within a range of roughly 8 to 17 ms. The saturation at large inputs reflects the refractory period of Section 2.3.3: however fast the membrane is recharged, a new spike cannot occur before h and n have partly recovered. The example traces in the lower chart illustrate the shorter intervals and the slightly smaller spike amplitude at strong inputs.

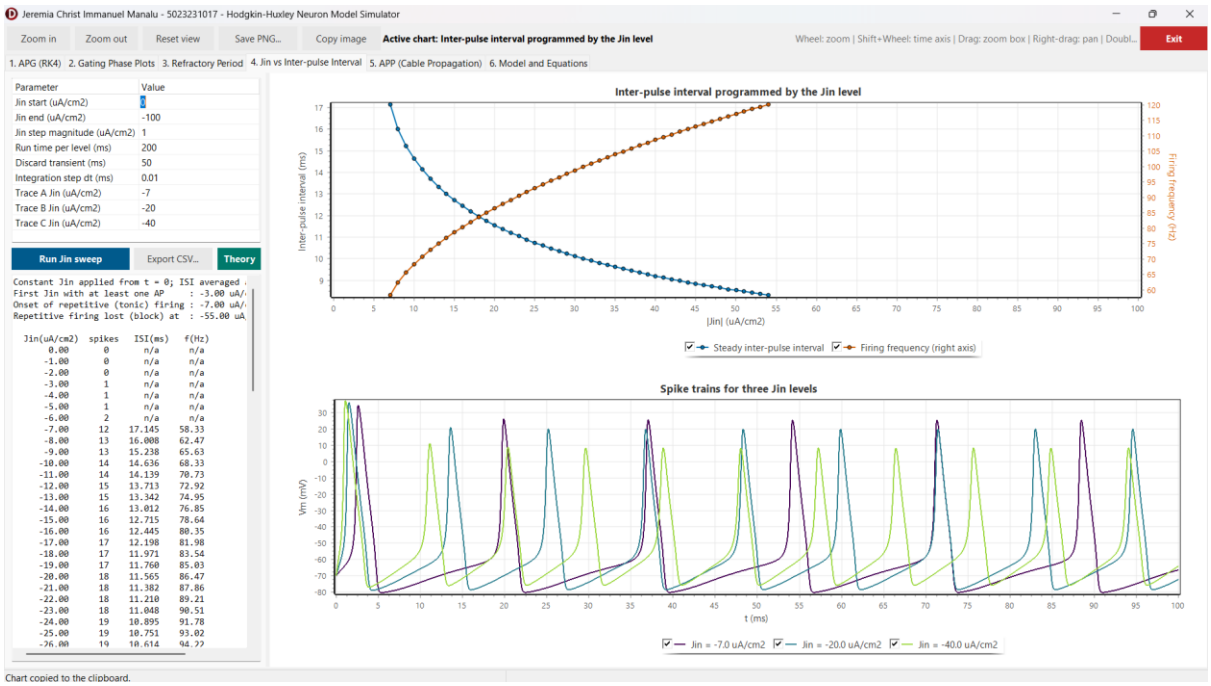


Figure 2.6. *Tab 4: steady inter-pulse interval and firing frequency against $|J_{in}|$, and spike trains for three input levels*

Table 2.4. *Steady inter-pulse interval and firing frequency for selected input levels (program export)*

J_{in} (µA/cm²)	Mean ISI (ms)	Frequency (Hz)
−6	- (2 spikes, no steady firing)	-
−7	17.15	58.3
−10	14.64	68.3
−15	12.72	78.6
−20	11.57	86.5
−30	10.13	98.7
−40	9.21	108.6
−54	8.33	120.1
−55	depolarization block	-

2.3.5 Action Potential Propagation

Figure 2.7 shows tab 5 for the active squid-axon cable. The result memo confirms the cable constants derived in Section 2.2.3 ($r_i = 19{,}893$ Ω/cm, $c_m = 0.14954$ µF/cm, $D = 0.33616$ cm²/ms, $r = 0.3362$). The Vm(t) chart shows the same action potential arriving at successive recording sites with a constant delay, and the Vm(x) chart shows the wave as a pulse that moves along the axon with an unchanged shape. The 0 mV crossing times grow linearly with distance, from 0.652 ms at the stimulated end through 1.486 ms at 12 mm and 2.980 ms at 30 mm to 4.476 ms at 48 mm, which gives a conduction velocity of 12.04 m/s at 6.3 °C. The peak voltage is 33.03 mV along the whole middle part of the axon. It is slightly higher at the stimulated end (34.07 mV), where the stimulus adds to the membrane current, and at the sealed far end (37.32 mV), where no axial current can flow onward and the charge accumulates in the last segments. The spatial extent of the action potential, about 2 to 3 cm, is consistent with the product of the velocity and the spike duration.

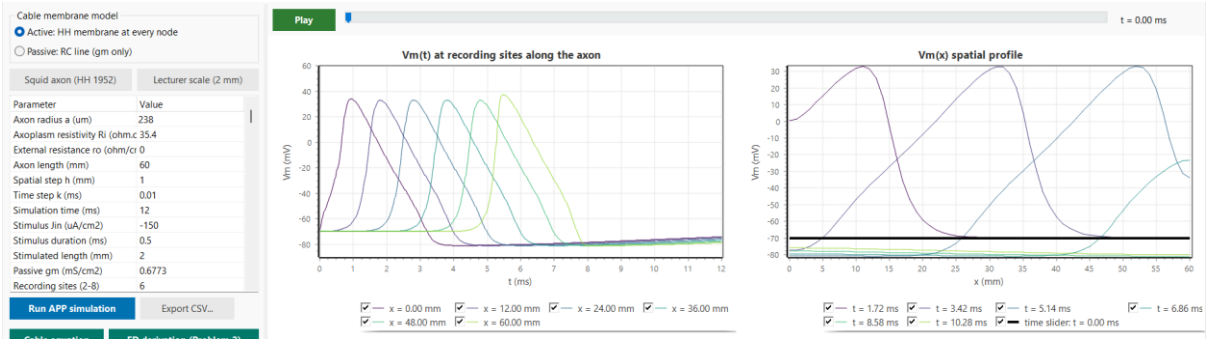


Figure 2.7. *Tab 5, active cable with the squid-axon preset: parameters, Vm(t) at six recording sites and Vm(x) at six instants*

Table 2.5 summarizes the parameter variations. Halving the spatial step to 0.5 mm (with a smaller time step to keep $r \leq 0.5$) changes the velocity by less than 2 %, which agrees with the second-order spatial accuracy of the scheme. Raising the temperature to 18.5 °C increases the velocity to 18.2 m/s, against 18.8 m/s calculated and 21.2 m/s measured by Hodgkin and Huxley [1], the agreement with their calculation is within about 3 %. In the passive mode no action potential appears: the depolarization decays from -35.2 mV at the stimulated end to -68.1 mV at 15 mm, in line with the computed length constant of 7.0 mm, which demonstrates that propagation requires the regenerative sodium current. The lecturer-scale preset (2 mm fibre of 1 μm radius) propagates at about 0.78 m/s, consistent with $\theta \propto \sqrt{a}$ and with the time scale of the lecturer's plots. A step larger than k_{max} is rejected by the program with the message predicted by Equation (2.21). Figure 2.8 shows the space-time behaviour in the 3D views, where the propagating action potential appears as a straight ridge in the (x, t) plane whose slope is the inverse of the velocity, and Figure 2.9 shows the derivation window of the cable equation opened from the same tab.

Table 2.5. *Propagation results for parameter variations*

Case	**Result**
Active cable, squid axon, 6.3 °C, $h = 1$ mm, $k = 0.01$ ms	$\theta = 12.04$ m/s, peak 33.03 mV, $r = 0.336$
Active cable, $h = 0.5$ mm	$\theta = 12.25$ m/s
Active cable, 18.5 °C	$\theta = 18.2$ m/s (HH 1952: 18.8 calculated, 21.2 measured)
Passive cable, $g_m = 0.6773$ mS/cm²	no AP, -35.2 mV at 0 mm to -68.1 mV at 15 mm, $\lambda = 7.0$ mm
Lecturer scale, $a = 1$ μm, 2 mm, $h = 0.1$ mm	$\theta \approx 0.78$ m/s, $r = 0.14$

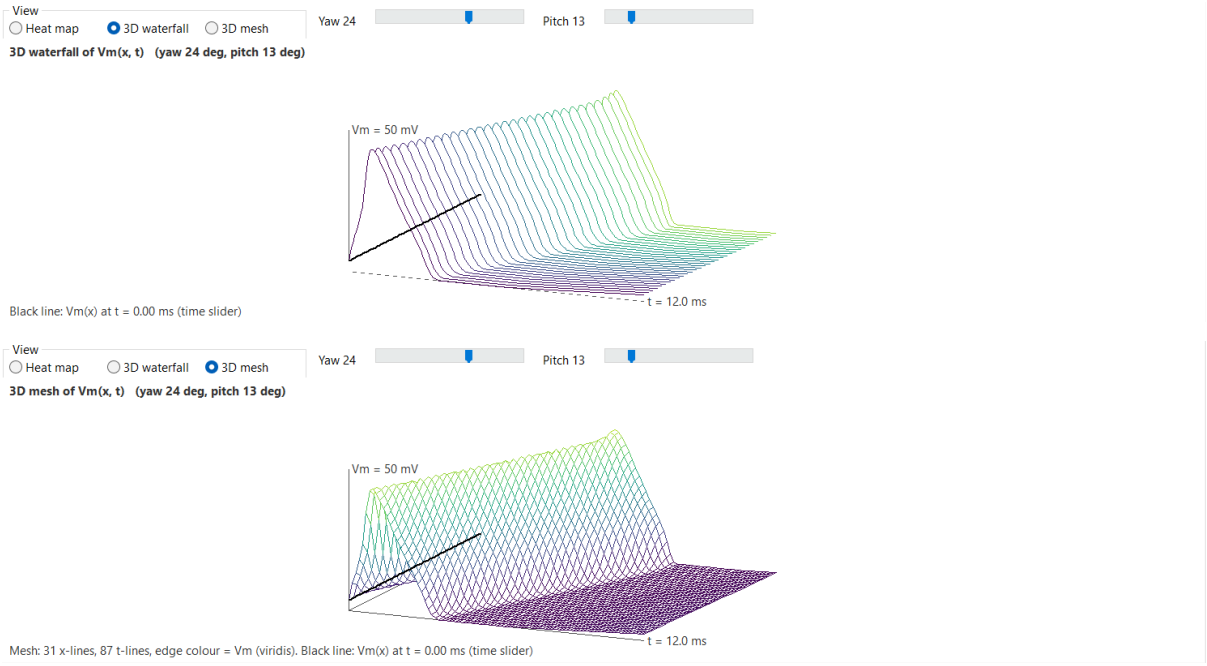


Figure 2.8. *Space-time views of $V_m(x,t)$ for the active cable: (a) 3D waterfall, (b) 3D mesh, the bold line is the spatial profile at the time-slider position*

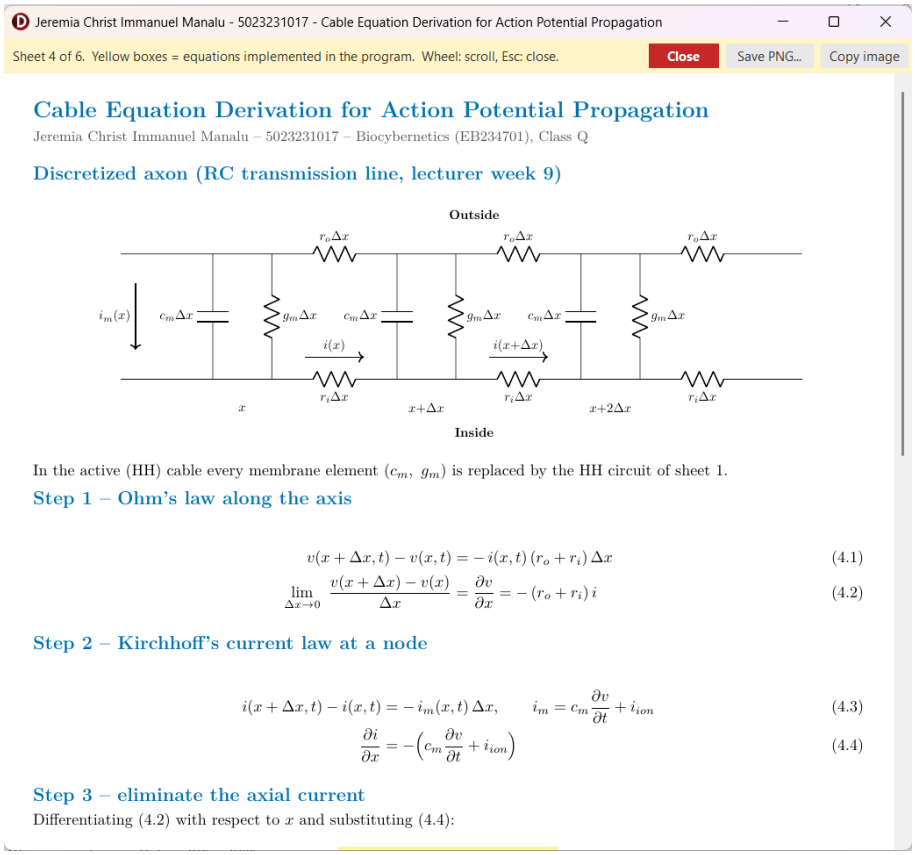


Figure 2.9. *Derivation window of the cable equation (sheet 4) opened from the main form, the yellow boxes are the equations used by the cable solver*

Compared with the reference programs, the results differ in two respects that are intended. The senior's propagation program updates the voltage with a pure diffusion term and an arbitrarily chosen coefficient applied to a stored APG trace, without any membrane current,

so its signal can only spread and decay, the active cable of this work regenerates the action potential at every node and therefore propagates without decrement, which is what Problem 3 asks the model to exhibit. The lecturer's and the senior's plots use a 2 mm axon, which corresponds to the lecturer-scale preset, the squid-axon default was chosen because its velocity can be checked against the 1952 paper.

2.4 Evaluation and Discussion

The results agree with the theoretical baseline of Chapter I in every aspect examined. The voltage equation (1.2) with the currents (1.3) and the kinetics (1.5) to (1.9) reproduces the resting equilibrium exactly and generates action potentials of the expected amplitude and duration once the output is aligned with Equation (2.2). The phase plots make the separation of time scales of Figure 1.2 directly visible: the fast gate m tracks its steady state, while the slow gates h and n lag behind and recover only gradually, and this lag is the common cause of the two behaviours of Problem 1. It produces the absolute refractory period of about 4 ms and the relative refractory period ending near 15 ms, and it bounds the inter-pulse interval from below, so that the input level can program the interval only between about 17 and 8 ms before depolarization block sets in. The supernormal phase found after the relative refractory period is not an artefact of the threshold search, which is monotone and resolved to 0.008 µA/cm², but a property of the HH kinetics that follows from the different recovery rates of h and n.

For propagation, the derivation of Problem 2 and the program of Problem 3 are consistent: the cable constants and the stability number printed by the program equal the hand-calculated values, the velocity converges with second-order behaviour when h is halved, and the comparison of the active and passive modes isolates the regenerative sodium current as the reason why an action potential propagates without decrement while a passive signal decays within a few length constants. The velocity of 18.2 m/s at 18.5 °C is within about 3 % of the value calculated by Hodgkin and Huxley [1], the remaining difference to their measurement of 21.2 m/s is of the same size as the difference between their own calculation and measurement, and it is expected because only the rate constants are temperature-scaled and the axon is modelled as a uniform cylinder.

Several limitations must be kept in mind. The explicit FTCS scheme is only first-order accurate in time and is stable only for $r \leq 1/2$, so fine spatial grids require small time steps, an implicit (Crank-Nicolson) scheme would remove this restriction. The sealed-end boundary raises the peak at the far end, and spikes are defined by a 0 mV crossing, so graded responses below 0 mV are not counted. Four discrepancies in the lecture material were identified and handled explicitly: the time step printed as "10 sec" (10 µs), the term $0.1(v + 2.5)$ printed in α_m instead of $0.1(v + 25)$, the coefficient $(r_o + r_i)C_m^{-1}$ instead of $[(r_o + r_i)c_m]^{-1}$, and the node index of the leakage term in the transcribed propagation code. In each case the program follows the derivation and the original paper, and offers the lecturer's integration method for direct comparison.

CHAPTER III. CONCLUSION

A Hodgkin-Huxley neuron simulator was developed in Delphi 13 with a numerical core that is independent of the user interface. The action potential is generated by integrating the four HH equations with the classical RK4 method (and, for comparison, with the lecturer's sequential RK4) at a step of 0.01 ms, and the output is aligned to the membrane voltage with $V_m = -(v + 70)$ mV, which turns the downward spikes of the 1952 sign convention into physiological action potentials from -70 mV to about $+35$ mV. A paired-pulse threshold search shows an absolute refractory period of about 4 ms and a relative refractory period ending near 15 ms, and a constant input programs the inter-pulse interval from about 17 ms (58 Hz) at -7 μA/cm² to about 8 ms (120 Hz) at -54 μA/cm², beyond which depolarization block occurs.

The integration of the cable equation was derived with a central difference in space and a forward difference in time, giving the explicit update $v_j^{n+1} = v_j^n + r\left(v_{j+1}^n - 2v_j^n + v_{j-1}^n\right) + k(J_{in} - J_{ion})/C_m$ with $r = k/[(r_o + r_i)c_m h^2]$ and the stability condition $r \leq 1/2$. Implemented with an HH membrane at every node, the scheme shows a regenerating action potential that travels at constant amplitude and at 12.0 m/s for the squid axon at 6.3 °C (18.2 m/s at 18.5 °C, close to the 18.8 m/s calculated by Hodgkin and Huxley), whereas the passive cable only produces a decaying depolarization. Propagation is therefore exhibited by the combination of axial current flow and the regenerative sodium current.

The two reviewed papers show the reach of the Hodgkin-Huxley formalism: extended with more than a dozen conductances, it explains the bursting of the respiratory pacemaker, and approximated with base-2 functions, it runs in real time on small FPGAs for neuromorphic and neuroprosthetic applications. All equations used by the program, including the derivation of Problem 2, are available in the program as separate sheets and are highlighted in this report.

REFERENCES

[1] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *J. Physiol.*, vol. 117, no. 4, pp. 500-544, Aug. 1952, doi: 10.1113/jphysiol.1952.sp004764.

[2] A. Arifin, "Hodgkin-Huxley neuron model," lecture notes, Biocybernetics (EB234701), Dept. Biomed. Eng., Inst. Teknologi Sepuluh Nopember, Surabaya, Indonesia, 2026.

[3] A. Arifin, "Hodgkin-Huxley neuron model simulation," lecture slides and source code, Biocybernetics (EB234701), Dept. Biomed. Eng., Inst. Teknologi Sepuluh Nopember, Surabaya, Indonesia, 2026.

[4] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2015.

[5] J. Malmivuo and R. Plonsey, *Bioelectromagnetism: Principles and Applications of Bioelectric and Biomagnetic Fields*. New York, NY, USA: Oxford Univ. Press, 1995.

[6] K. W. Morton and D. F. Mayers, *Numerical Solution of Partial Differential Equations: An Introduction*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2005.

[7] B. Amini, A. Bidani, J. B. Zwischenberger, and J. W. Clark, Jr., "A model of the pacemaking neuron of the respiratory central pattern generator," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 13, no. 2, pp. 120-124, Jun. 2005, doi: 10.1109/TNSRE.2005.847379.

[8] S. Haghiri, A. Naderi, B. Ghanbari, and A. Ahmadi, "High speed and low digital resources implementation of Hodgkin-Huxley neuronal model using base-2 functions," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 1, pp. 275-287, Jan. 2021, doi: 10.1109/TCSI.2020.3026076.

[9] D. Noble, "A modification of the Hodgkin-Huxley equations applicable to Purkinje fibre action and pace-maker potentials," *J. Physiol.*, vol. 160, no. 2, pp. 317-352, Feb. 1962, doi: 10.1113/jphysiol.1962.sp006849.

[10] S. Buccelli *et al.*, "A neuromorphic prosthesis to restore communication in neuronal networks," *iScience*, vol. 19, pp. 402-414, Sep. 2019, doi: 10.1016/j.isci.2019.07.046.

[11] T. Levi, P. Bonifazi, P. Massobrio, and M. Chiappalone, "Editorial: Closed-loop systems for next-generation neuroprostheses," *Front. Neurosci.*, vol. 12, Art. no. 26, Feb. 2018, doi: 10.3389/fnins.2018.00026.